MLFF Training-Data and Fine-Tuning Architecture

Purpose and authority

This manual defines the accepted current scientific, statistical, execution, and evidence architecture for the mdstats MLFF workflow: source-certified atomistic data preparation, leakage-safe evidence roles, neutral statistical preparation, one optional automatic target-size diagnostic feeding an operator-owned target-size decision, MACE fine-tuning, selected-only method validation, fresh final production, and bounded campaign execution. Downstream deployment, physical, calibration, and locked-test capabilities remain separately owned product obligations.

It is intentionally present-tense and single-generation. A reader does not need release chronology, migration history, or obsolete stage semantics to determine current behavior.

The canonical editable architecture sources are the numbered chapters under `docs/arch_manuals/mlff_training_data/`. The assembled Markdown and PDF are generated publication products of those sources and must not be edited as independent authorities.

Detailed exact behavior is owned by current specifications under `docs/specs/training_data/`. Methods/theory material may explain rationale but does not override architecture or specifications. Proposed transitions live in `workplans/`; completed chronology lives under `docs/history/mlff/`; correctness/performance evidence lives in audits, release evidence, and benchmarks.

Architectural motive

MLFF campaigns combine state with fundamentally different epistemic roles: physical source facts, eligibility decisions, evidence partitions, fitted transforms, subset-membership decisions, target-size decisions, optimization/checkpoint state, protocol-validation evidence, calibration evidence, locked tests, and deployment decisions. Conflating those roles creates leakage and ambiguous authority even when the numerical code is correct.

The architecture therefore uses immutable/content-addressed evidence, explicit statistical roles, one normative owner per scientific decision, and authenticated dependency direction. Execution realization is kept separate: cache layout, worker count, queue order, out-of-core storage, and scheduler policy may change without changing scientific membership, ordering, coverage, ranking, or evidence roles.

Expensive exact numerical work is computed once per semantic identity and reused wherever its inputs are unchanged. Exactness, deterministic authoritative decisions, bounded materialization, explicit resource ownership, and restartable authenticated state take precedence over nominal utilization.

Current workflow at a glance

```
source evidence and labels
  -> eligibility / physical conditions
  -> raw feature and event evidence
  -> evidence-role partitioning
  -> neutral statistical substrate and protected relations
  -> fitted descriptors, metrics, E0/objective/weight inputs
  -> one P_train / M3 target-size development split
  -> one canonical training order pi_train and evaluation ladder M1 subset M2 subset M3
  -> one common deterministic target-size preparation
  -> optional paired optimizer-seed automatic diagnostic over candidate sizes
     (one target-size reducer -> a *recommended* size)
  -> operator-owned provisional design (ordered collection of (N, CV horizon, production
horizon))
  -> cross-validate admission
  -> frozen design: every selected size N_selected, its exact T_selected =
pi_train[:N_selected], and role horizons
```

```
-> post-selection cross-validation on the frozen collection
-> fresh final production on the selected dataset(s)
-> currentness-fenced final-production publication
```

The current graph has exactly one target-size architecture. The retired per-domain multi-view selection generation is not an alternate current path: it is neither migrated nor semantically read forward, and a workspace still holding its derived state is rejected with an actionable destructive reset/reprepare requirement before any candidate, checkpoint, or descendant is reused. Raw scientific inputs and independently valid low-level content caches remain reusable when their recipes do not depend on retired target-size semantics.

Reading index

Need	Primary chapter
Scientific motivation, record/evidence model, and scope	Part I - Foundations
Source identity, labels, strain/stress, eligibility, raw features/events	Part II - Data and evidence contracts
Evidence roles, leakage-safe CV, fitted preparation, objective/weighting/exposure boundaries	Part III - Statistical design and fitted preparation
Replay, MACE protocol, checkpointing, validation, deployment, calibration, active learning	Part IV - Training, evaluation, and deployment
Target-size split/orders, the optional paired-seed diagnostic and its reducer, the provisional design and its freeze, post-selection CV, fresh final production	Part V - Target-size selection and post-selection validation
Exact execution, bounded resource/materialization, cache/restart/storage/progress	Part VI - Performance and execution architecture
Sole-owner matrix and accepted extension boundaries	Part VII - Ownership and extension boundaries
External scientific/algorithmic sources	References

Context retrieval index

For targeted human or AI loading, use the smallest current source containing the needed concept:

Query terms	Load first
source/label identity, eligibility, strain/stress, raw features/events	`20_data_contracts.md`
evidence roles, leakage, CV, fitted metrics, E0, objective, weighting, exposure	`30_statistical_design.md`
replay, MACE, checkpoint, evaluation, deployment, calibration, active learning	`40_training_evaluation.md`
target size, `pi_train`, `T_selected`, `M1/M2/M3`, `n1/n2/n3`, post-selection CV, final production	`50_target_size_selection.md`
scheduler, sparse execution, out-of-core, memory, persistence, progress	`60_execution_performance.md`
owner, dependency direction, unsupported generation, extension boundary	`80_ownership_and_decisions.md`
scientific/algorithmic provenance	`90_references.md`
superseded design rationale or release chronology	`docs/history/mlff/`
proposed transition	`workplans/active/`

Stable terminology

- **training domain** — an authorized gradient-training evidence partition. The target-size design is an ordered frozen collection; for each frozen size, post-selection CV may derive fold-local partitions only inside its exact membership `T_N = pi_train[:N]`.
- **target membership** — frame membership in a target-training subset; an exact prefix of the one canonical training order `pi_train`.
- **target size** — the protocol-level scientific target-training cardinality the operator chooses, restricted to the configured qualified candidate set.
- **recommended size** — the size the optional automatic diagnostic's reducer ranks best under its short-horizon protocol. It is evidence, never authority.
- **monitor size** — the cardinality of a monitoring/evaluation evidence set; never target-size authority.
- **training order** — the one canonical deterministic ordering `pi_train` of the target-training pool whose prefixes define candidate target subsets.
- **qualified size** — a candidate size admitted by the configured target-size policy for the current experiment definition.
- **provisional design** — the ordered, unique-by-`N` collection of per-size entries `(N_provisional, its exact membership, selection source, CV horizon, production horizon)` the operator owns until admission. Empty is its canonical unselected state.
- **selected size** — a target size `N_selected` in the ordered frozen design admitted at `cross-validate`, bound to its exact membership `T_selected = pi_train[:N_selected]` and its effective role horizons.
- **authoritative evidence** — persisted information that defines or independently proves a scientific decision.
- **reconstructible execution cache** — discardable state derivable exactly from authoritative inputs.
- **unsupported generation** — an old campaign/artifact generation that current architecture does not interpret or migrate; it requires re-preparation.

Normative vocabulary

- **SHALL / MUST** — required for scientific, statistical, or execution correctness.

- **SHOULD** — the default design unless measured evidence justifies another exact-equivalent realization.
- **MAY** — optional realization that cannot weaken the scientific contract.

When architecture explains a change-sensitive constant whose exact value is specification-owned, the owning specification remains the sole normative location for changing that value.

Retrieval and local-context rule

Each major chapter states what its concepts own, consume, emit, and explicitly do not own. Equations and symbols are defined near first use. A chapter may repeat a dependency boundary for local comprehension, but repeated prose must not create a second independently tunable contract.

Part I - Foundations

What an MLFF learns

An energy-conserving machine-learned force field represents a potential-energy function

$$E_\theta = E_\theta(\mathbf{Z}, \mathbf{R}, \mathbf{H}),$$

where $\mathbf{Z}$ contains atomic numbers, $\mathbf{R}$ positions, $\mathbf{H}$ the periodic cell, and θ model parameters. Forces and stress follow from derivatives of the same energy,

$$\mathbf{F}_i = -\frac{\partial E_\theta}{\partial \mathbf{R}_i}, \qquad \boldsymbol{\sigma} = -\frac{1}{V}\frac{\partial E_\theta}{\partial \boldsymbol{\epsilon}},$$

under the declared stress sign and strain convention of the label source. MACE constructs symmetry-aware local atomic representations and sums atomic-energy contributions [1]. A useful training/evaluation corpus must therefore constrain both the energy surface and its derivatives throughout the intended simulation domain.

A low global force error is not sufficient. Common framework vibrations can dominate aggregate statistics while rare mobile-ion environments, strain states, migration geometries, interfaces, defects, or other declared focus physics remain poorly represented. The architecture separates broad numerical metrics, condition/group-resolved evidence, physical-observable validation, and explicit extrapolation/challenge evidence. The current P6 campaign ends at selected-only method validation and fresh final production; downstream qualification is separately activated.

Why trajectory frames need statistical roles

Molecular-dynamics frames are temporally correlated. Neighboring configurations can be near duplicates, so assigning them to nominally different roles can create leakage and overstate model quality.

For observable x_t, normalized autocorrelation at lag k is

$$\rho_x(k) = \frac{\langle (x_t - \bar{x})(x_{t+k} - \bar{x}) \rangle}{\langle (x_t - \bar{x})^2 \rangle}.$$

A truncated integrated autocorrelation time is

$$\tau_{\mathrm{int},x} = \Delta t \left[\frac{1}{2} + \sum_{k=1}^{k^\star} \rho_x(k) \right],$$

with approximate effective sample count

$$N_{\mathrm{eff},x} \approx \frac{T}{2\tau_{\mathrm{int},x}}.$$

mdstats therefore uses autocorrelation-aware complete-frame blocks, purge semantics, and explicit independence grades rather than treating every frame as independent [3-5]. Exact estimators, truncation, block size, purge, and role-assignment rules are specification-owned.

Evidence-role model

The architecture distinguishes evidence by what it is allowed to control.

Role	Supplies gradients?	May control fitted preparation/subset/size/checkpoint?	Purpose
development / training domain	Yes when selected	Yes, within the authorized training/model-selection contract	fitting and protocol development
checkpoint / common target monitor	No	Yes, only for explicitly authorized development/model-control decisions	stopping/checkpoint and target-size development evidence
held-out CV evaluation	No	No for the frozen protocol it evaluates	protocol validation
calibration	No	No training/subset/checkpoint changes	final-committee uncertainty calibration
locked interpolation/ challenge test	No	No	sealed final evaluation

Calibration is not test data; held-out CV is not a checkpoint monitor; and a monitor cardinality is not a target-training cardinality.

Scope and ownership

The MLFF subsystem owns dataset certification, evidence-role construction, fitted preparation, one global target-size study, training-artifact construction, current campaign orchestration, checkpoint/evaluation lineage, and active-learning lineage. Downstream deployment, physical, calibration, and locked-test consumers retain their product obligations without becoming P6 selection owners.

Its current responsibilities include:

- VASP source discovery/certification and source/label identities;
- composition, thermodynamic condition, ensemble, reference-cell, strain/stress reconstruction;
- electronic-structure compatibility and label-domain grouping;
- energy/force/stress audit and atomic-reference identifiability/fitting lineage;
- immutable frame facts, eligibility, and quality decisions;
- generic raw structural features/events plus explicit optional material/profile extensions;
- autocorrelation-aware complete-frame blocks and role feasibility;
- fixed outer roles and independent CV job families;
- neutral and authorized fold-local fitted descriptors, transforms, metrics, E0, objective/weight, and difficulty evidence;
- the target-size development split, the canonical training/evaluation orders, the common preparation, and the paired optimizer-seed screen;
- the ordered frozen target-size collection, its exact per-size prefix memberships, and role horizons;
- MACE target/replay artifacts and explicit exposure realization;
- replay-retention and checkpoint admissibility;

- post-selection protocol-matched CV and fresh final training; downstream committee, calibration, sealed evaluation, and deployment verification are separate consumer boundaries;
- active-learning candidate/DFT lineage where supported by current specifications.

The subsystem does not silently merge incompatible electronic-structure levels, infer ambiguous scientific references, use held-out/locked evidence for forbidden model-control decisions, redefine analysis-owned physical-observable algorithms, create a second target selector, generate rescue target sizes, or migrate unsupported old campaign generations.

LTA/zeolite ring, cage, site, crossing, and related semantics are optional profile extensions rather than generic defaults.

Reference application: Li/Na/K-LTA

The principal reference application contains AIMD evidence spanning multiple cation compositions, temperatures, and strain conditions. It motivates—but does not hard-code into generic architecture—several requirements:

1. framework atoms can outnumber mobile cations, so aggregate metrics must not hide declared mobile-species environments;
2. strain conditions need not form a full Cartesian product with composition/temperature, so condition applicability may be hierarchical;
3. one trajectory per condition supplies limited independence and must not be represented as an independent-replica test;
4. fixed framework stoichiometry can make individual atomic reference-energy corrections non-identifiable without anchors;
5. short trajectories may contain few rare transitions, so absent events are explicit coverage gaps rather than evidence of irrelevance.

Reuse of analysis and sampling capabilities

The MLFF workflow orchestrates existing mdstats capabilities instead of duplicating them.

Capability	MLFF use
`mdstats.io.vasp.read_vasp_frames`	cells, coordinates, energies, forces, stress, temperature, provenance
VASP control/ensemble readers	controls, energy-channel and ensemble evidence
trajectory-quality / production-regime assessment	source and stationary-regime evidence
analysis structural/topology modules	optional profile-owned raw evidence or post-training observables under analysis contracts
sampling/cross-fit primitives	source-bound blocks, purge, and independence semantics

Physical observables remain owned by `mdstats.analysis`. The MLFF layer may orchestrate matched evaluation and retain analysis-owned result identities, but it does not redefine RDF, MSD, VACF, VDOS, diffusion, topology, conductivity, or related numerical algorithms.

Current controlling data flow

```
source bytes / controls / trajectory collections
  -> source and label-domain certification
  -> immutable frame facts and eligibility
  -> raw features/events before ordinary thinning
```

```
  -> correlation-aware blocks and evidence-role feasibility
  -> development / monitor / post-selection CV roles
  -> neutral DATA6/DATA7 fitted preparation
  -> P_train / M3 split -> pi_train / pi_eval
  -> common target-size preparation
  -> optional paired optimizer-seed diagnostic -> target-size reducer
  -> target-size study using authorized development/model-selection evidence,
     yielding a recommendation rather than a decision
  -> operator-owned provisional design, frozen at cross-validate admission
  -> ordered collection of frozen entries (N_selected, exact T_N = pi_train[:N_selected], and
role horizons)
  -> for each frozen size, protocol-matched CV partitions inside its exact T_N, with held-out
folds inaccessible to size/checkpoint choice
  -> accepted frozen protocol
  -> independent final seeds and checkpoint admission
  -> current final-production publication
  -> separately activated downstream committee/physical/calibration/locked consumers where
implemented
  -> active-learning lineage where configured
```

No allowed dependency runs from held-out CV or locked-test evidence backward into fitted transforms, E0 fitting, target membership, target-size selection, checkpoint choice, or calibration-policy design.

Responsibility separation is more durable than module layout

The implementation may reorganize Python modules while preserving the architecture. The durable separation is among:

- physical/source facts;
- evidence-role and policy decisions;
- training-domain fitted products;
- target-membership and target-size decisions;
- runtime/execution realization;
- validation/calibration/locked evidence;
- external analysis-owned results.

Current specifications control public/serialized current-generation contracts. Internal refactoring may reuse common sampling/execution primitives when externally owned scientific behavior and persisted current-generation identities remain conforming. Backward compatibility with superseded campaign generations is not an architectural requirement. The accepted current-generation P5A6 workspace remains a required unchanged reopen boundary; obsolete derived target-size generations are rejected before reuse and current preparation creates a fresh configurable authority.

Part II - Data and evidence contracts

Purpose and ownership

This chapter defines immutable source/frame facts, label-domain identity, physical conditions, quality/eligibility, raw feature/event providers, and correlation-aware complete-frame evidence blocks.

It does not own evidence-role assignment beyond the records needed to support DATA5, fitted statistics, target membership, target size, training exposure, checkpoint selection, or validation decisions.

Evidence records and immutability

The MLFF data model separates source facts, workflow decisions, fitted products, runtime realizations, and external scientific results. A new policy creates new policy/decision records rather than mutating immutable source/frame facts.

Source and frame facts

`TrainingDataSource` owns source occurrence identity, path/location hints, content hashes, composition/controls, ensemble/quality/production evidence, label-domain identity, and declared reference grouping.

`TrainingFrameRecord` owns source-bound frame facts such as `frame_uid`, source occurrence, frame index/time, atoms/cell, label references, physical conditions, and distinct geometry/label fingerprints.

`TrainingFrameRecord` does **not** own eligibility, statistical role, target membership, target size, training exposure, calibration, or acquisition state.

Decision, policy, fitted, and realization families

Representative downstream/current families include:

```
FrameEligibilityDecision
PartitionAssignment
CandidateAdmissibilityDecision
AcquisitionDecision

PartitionRoleBudgetPolicy
PartitionFeasibilityReport
FeatureMetricPolicyTemplate
FoldFeatureMetricFit
FinalFeatureMetricFit
TrainingObjectivePolicy
ConfigurationWeightPolicy
PropertyWeightPolicy
CheckpointMetricPolicy
TargetSizeExperimentDefinition
ResolvedTargetSizePolicy
TargetSizeCommonPreparation
TrainingProtocolIdentity
MaceCheckpointControlPolicy
ExposureBackendPolicy
MaceExposureRealizationRecord
ReplayRetentionPolicy
ProtocolFreezeRecord
CalibrationApplicabilityDomain
CalibrationTransferDecision
```

A static policy defines an algorithm and fixed choices. A fitted record contains parameters learned from one explicitly authorized training domain. A realization record records behavior actually observed from an external/runtime system. Those roles are not interchangeable.

Target membership is intentionally absent from the generic DATA2-DATA4 record list because its current authority is the canonical training order `pi_train` derived by the target-size owners in Part V.

Digests and signatures

Deterministic content/policy/source digests bind identity and detect modification. They do not by themselves authenticate authorship. Serialized current records carry version/schema and deterministic content identity under their owning specifications.

Source manifest and occurrence identity

A review/production manifest supplies source locators, grouping declarations, scientific assertions that cannot be reconstructed unambiguously from one source file, and explicit expert overrides with rationale. Directory/file naming is diagnostic input, not accepted physical truth without verification.

The source byte/content identity is distinct from a manifest occurrence. Byte-identical copies may share a source-content identity while deliberately distinct manifest runs have distinct occurrence identities.

A frame occurrence derives from occurrence identity plus source frame index. This keeps occurrence identity stable across later concatenation/export while permitting duplicate-geometry detection across separate source occurrences.

Geometry, label, and labeled-configuration identities

The architecture keeps three identities separate:

```
geometry_fingerprint
label_payload_digest
labeled_configuration_fingerprint
```

`geometry_fingerprint` identifies atomic geometry independently of energy/force/stress labels under the current canonical wrapping/cell/tolerance policy. `label_payload_digest` binds the selected labeled payload and label-domain identity. `labeled_configuration_fingerprint` combines geometry and label payload.

Leakage auditing may use occurrence overlap, exact geometry overlap, exact labeled-configuration overlap, declared near-geometry/descriptor criteria, restart/copy detection, and forbidden temporal proximity. Approximate or symmetry-aware matching may exist only under an explicit current policy; it cannot change the semantic roles above.

Electronic-structure label domains

Electronic-structure identity is decomposed because not every input difference has the same scientific meaning:

```
TheoryIdentity
EnergyReferenceIdentity
DerivativeConvention
NumericalQualityProfile
SoftwareProvenance
```

A versioned `LabelCompatibilityPolicy` classifies differences as compatible, compatible with quality evidence, separate label domain, or unresolved. Theory- or reference-defining differences cannot be silently waived.

A target training bundle contains one compatible target label domain and, when enabled, a separately identified replay head/lineage. Incompatible target DFT levels produce separate target bundles rather than an implicit mixed target domain.

Energy channel

The selected target energy is an explicit named channel consistent with derivative labels. Its channel, units, completeness, electronic/reference convention, and provenance are preserved. Energy/force/stress labels that do not share an accepted derivative/reference convention are not silently combined.

Atomic-reference identifiability and fitting boundary

For elemental correction vector $\Delta\mathbf{e}_0$, the schematic fit is

$$\mathbf{A}\,\Delta\mathbf{e}_0 \approx \mathbf{b},$$

with configuration-element count matrix $\mathbf{A}$ and target-minus-foundation energy residual $\mathbf{b}$.

`AtomicReferenceIdentifiabilityReport` depends on elemental count support rather than fitted target residuals. It records element order, matrix shape/rank/singular values, condition/null-space information, identifiable combinations, outcome, and transfer limitations. It does not contain fitted elemental corrections.

The actual `AtomicReferenceFitRecord` is a DATA7 fitted object bound to one fold/final training domain, foundation checkpoint identity, identifiability report, solver/tolerance, elemental support, fitted corrections, residual, and policy outcome.

Each cross-validation fold receives its own fold-local fit. Final training receives a separate final-training fit. Monitors, calibration, held-out evaluation folds, and locked tests are excluded from the fit.

MACE export receives the exact accepted numerical E0 representation, normally an atomic-number mapping; a record/path name is provenance rather than an E0 payload.

Ensemble, temperature, cell, and strain

Ensemble and temperature

The subsystem consumes mdstats control/ensemble certification and distinguishes equilibrium, pressure-controlled, ramped/driven, multi-thermostat, and unresolved cases under the owning control specification. Ensemble is not inferred merely from observed cell variation.

Nominal temperature controls and realized ionic-temperature statistics remain separate. `TemperatureCondition` binds requested/thermostat targets, realized series/statistics, drift/stationarity evidence, and ramp status as applicable.

Reference-cell resolution

Strain requires an explicit or uniquely resolvable compatible reference. Accepted resolution order is controlled by current specifications and may use an explicit matrix/structure/run or a unique compatible unstrained member of the declared reference group. Ambiguity fails closed.

Cell and deformation convention

ASE cell vectors are rows. For fractional row vector $\mathbf{s}_{\mathrm{row}}$ and cell $\mathbf{H}$,

$$\mathbf{r}_{\mathrm{row}} = \mathbf{s}_{\mathrm{row}}\mathbf{H}.$$

For reference $\mathbf{H}_0$ and current cell $\mathbf{H}_t$, the reported deformation gradient acting on Cartesian column vectors is

$$\mathbf{F}_t = \left(\mathbf{H}_0^{-1}\mathbf{H}_t\right)^T.$$

An internal right-acting row-vector form is acceptable only when serialization/reporting returns the declared Cartesian-column convention. Rotation/stretch separation uses the declared polar-decomposition convention.

Stored strain evidence includes the applicable volume, linear/finite/logarithmic, hydrostatic/deviatoric, principal, shear, rotation, coordinate-frame, and storage-convention quantities. Qualification includes nonsymmetric shear and rotated-stretch cases so transpose/left-right errors cannot hide behind diagonal fixtures.

Hierarchical condition schemas

Condition space is not assumed to be a global Cartesian product. A material profile declares applicable condition axes and hierarchical strata. For example, an LTA profile may separate unstrained composition/temperature/regime strata from strained composition/reference-condition/strain-mode/sign/regime strata. Only observed and scientifically applicable combinations are required.

Stress and virial

Canonical `REF_stress` is a symmetric Cartesian Cauchy-stress tensor in eV/Angstrom^3 under the ASE/MACE sign convention qualified by the runtime lock. Tensor shear carries no engineering-factor multiplication. Intermediate Voigt ordering, when used, is explicit and round-tripped.

Virial and stress have distinct keys and are never silently relabeled. Qualification covers units, finite-strain sign, tensor/Voigt order, shear factors, and MACE read-back. Missing stress may carry zero stress weight only under an explicit heterogeneous-label policy.

Eligibility and quality

Run/source quality distinguishes qualified, degraded, unqualified, and unresolved states under current policy. Overrides are explicit evidence.

`FrameEligibilityDecision` applies after labels exist. Hard rejection includes absent/nonfinite required labels or geometry/cell, singular/corrupt structures, incomplete ionic records not recoverable under the current interruption policy, catastrophic overlaps, and disallowed electronic-convergence failures.

Soft evidence records transient regimes, unusual but physical forces/stress, rare coordination/events, topology changes, model residuals, and degraded numerical quality without turning percentile tails into automatic rejection.

Pre-DFT candidates use a separate `CandidateAdmissibilityDecision` over geometry/cell safety, element/count policy, topology/integrity, trajectory/integrator evidence, model outputs, and descriptor availability. After DFT labeling they re-enter normal source/frame eligibility lineage.

Material profiles and feature providers

Declarative profile boundary

`SystemProfileProvider` owns material identity: phases, geometry, chemistry modifiers, optional structural extensions, meaningful atom groups, condition axes, and independence axes. It does not itself own calculated scientific feature arrays.

Profiles are compositional rather than a single flat material enum. Interface/multiphase systems explicitly declare component membership. Generic fallback supplies only generic groups/axes; porous/zeolite/LTA semantics require the corresponding explicit extension chain and never activate automatically.

Universal structural selection inputs

The generic structural provider supplies selection-grade local geometry descriptors such as smooth chemistry-scaled coordination, support-neighbor count, radial projections, local-density/mixing proxies, angular moments, and rotationally invariant orientational-order summaries.

These are upstream target-subset inputs, not an independent selector and not replacements for analysis-owned RDF, integer coordination, full angle distributions, structure factors, or topology observables.

Descriptors aggregate by authorized atom groups and elements present in the permitted domain. Generic temporal events capture large local structural changes without assigning material-specific physical meaning.

Partition-critical profile features

Rare categorical states that partition policy promises to protect are available at full resolution before the outer partition freezes. A profile may supply phase, environment, defect, region, molecular, or event states.

Optional LTA state includes framework/mobile roles, resolvable ring/site class, off-center class, coordination/site/ring-crossing changes, and framework-integrity evidence. Unresolved required classifications produce explicit coverage/partition limitations rather than fabricated balanced strata.

Optional learned-model features

A qualified optional MACE provider may supply foundation/model identity, invariant atomic descriptors, group/species environment summaries, and authorized zero-shot prediction/residual evidence. MACE/PyTorch remain optional dependencies to the mdstats core.

Feature blinding and fitted metrics

Geometry-only descriptors from a frozen model may be computed wherever authorized. Label-derived residual/difficulty features may be exposed only inside their applicable training domain.

Outer monitor, calibration, held-out evaluation, and locked-test residuals do not enter feature fitting or target-subset construction. Evaluation predictions can be persisted in blinded catalogs without exposing residual-derived selector inputs. Violating this boundary is a hard leakage failure.

Raw feature providers are partition-independent. Dataset-dependent scaling/PCA/whitening/metric fitting is represented by separate static templates and fold/final fitted records. For fold k, fitting may inspect only its gradient-training domain; other domains may be transformed by the frozen fitted object but cannot influence it.

A block-normalized metric may take the form

$$d^2(i,j) = \sum_b w_b \frac{\| \mathbf{z}_i^{(b)} - \mathbf{z}_j^{(b)} \|^2}{d_b},$$

where block weights, dimensions, missing-block behavior, dtype, tolerance, and any fitted scaling/projection are explicitly identified. High-dimensional learned descriptors do not dominate solely because they contain more components.

Event detection before thinning

The controlling order is:

1. source/label integrity on full-resolution frames;
2. event/change detection on all eligible frames;
3. protected event-window preservation;
4. temporal thinning of the ordinary non-event pool;
5. higher-cost descriptor/fitted target-subset-input operations.

Event stencils are policy-controlled and compact by default. Adjacent frames from one physical event are not mistaken for independent rare-event evidence.

Autocorrelation and complete-frame blocks

Fast observables determine the minimum ordinary decorrelation/block scale; slow structural variables diagnose whether state-level independence exists at all. Candidate stride is defined in physical/frame time from the declared fast autocorrelation estimate and applies only to the non-event pool.

A `TrainingDataBlock` is a contiguous interval of whole configurations with block/run/frame bounds, represented time, regime, correlation evidence, and configuration identities. Atoms from one configuration are never split across statistical roles.

A purge interval separates roles using the declared physical/autocorrelation/event/restart policy. If a slow state never decorrelates, the independence report explicitly states that temporal blocking does not provide state-level independence.

Part III - Statistical design and fitted preparation

Purpose and ownership

This chapter defines the evidence roles and fitted-preparation boundary that make target-size comparisons and later method validation interpretable. It owns independence, protected relations, leakage boundaries, fitted products, objective/weighting inputs, and the distinction between development evidence and later validation roles.

It does **not** own target membership or target size. The Part V owners derive one `P_train`/`M3` split, one canonical `pi_train`, and one target-size result. After admission, the Part V/P5 owners may partition each frozen size's exact membership `T_N` for cross-validation; that operation cannot choose a new size or membership.

Independence and evidence roles

Evidence uses the strongest available independence level, for example:

1. independent replica/velocity seed or independently prepared realization;
2. independent structural or chemical ordering;
3. independent thermodynamic run;
4. a purged temporal block within one run.

Temporal separation does not create an independent metastable state when the relevant slow variable has not decorrelated. Every cohort carries machine- readable independence evidence and known limitations.

Before roles are assigned, the partition policy declares requested cohorts, minimum independent blocks, purge requirements, protected relations, and allowed reductions. A feasibility report may therefore record full support, temporal-block-only support, deferred calibration, external-only challenge evidence, a reduced fold count, or insufficient support. The workflow never fabricates a role from a short or correlated trajectory to satisfy a percentage target.

The current development evidence roles are:

```
development_pool
common_target_monitor
post_selection_cv_folds
```

The broader product architecture reserves separate calibration and locked-test roles for downstream qualification. Those consumers are not part of the P6 campaign lifecycle and their absence is not converted into current selection or production evidence.

Only the development pool supplies gradient-training candidates. The common target monitor is development/model-selection evidence: it may control the authorized target-size screen and post-selection checkpoint policy, but it supplies no gradients and is not a held-out CV fold.

One global selection universe

The neutral statistical substrate supplies duplicate groups, correlation families, provenance relations, and split exclusions before target-size selection exists. It produces exactly one development split:

```
eligible labelled frames -> P_train (target-training pool) + M3 (development monitor)
```

`P_train` is ordered once as `pi_train`; the target-size owner defines every candidate as an exact prefix. `M3` is ordered once as `pi_eval`; `M1`, `M2`, and `M3` are direct nested evaluation populations. No complement, per-domain membership map, or alternate ordering may change the universe.

Protected relations remain intact wherever the current owner assigns roles. An inseparable duplicate/correlation component cannot be split merely to obtain a requested fold count. For any frozen size, a frame outside its exact

membership T_N cannot enter post-selection CV because it is convenient or because it belongs to a related source cohort.

Cross-validation validates a frozen protocol

Target size is frozen before protocol-matched cross-validation is interpreted. For each required post-selection fold (k) of a frozen size N, the owner keeps distinct:

```
fold_training_partition_k within T_N
fold_checkpoint_monitor_k
held_out_evaluation_partition_k within T_N
```

For each frozen size, its cardinality N and exact membership T_N remain unchanged across folds. Fold assignment is local to T_N, and fold-local fitted preparation may use only that fold's training partition and authorized monitor. It may not inspect the held-out partition, outer protected evidence, or locked evidence before checkpoint choice. The final fold evaluation occurs only after the fold representative is frozen.

This gives the required distinction:

```
target-size admission    -> ordered frozen design of selected sizes N and exact memberships T_N
post-selection CV        -> method validation on the frozen design
```

Held-out CV error, calibration evidence, and locked-test evidence therefore cannot select or alter the frozen target design, or tune the target-size policy.

Fitted preparation

The current common preparation is built once from the neutral substrate and the frozen foundation/training protocol. It may emit:

- descriptor coordinates and fitted feature metrics;
- foundation predictions and training-domain residual/difficulty evidence;
- atomic-reference/E0 fits;
- objective, configuration-weight, and property-weight records;
- condition, provenance, event, environment, and diversity inputs;
- deterministic identities binding each product to its authorized inputs.

These products are inputs to the one canonical training order. They are not a second selector. A fitted transform, metric, residual, or E0 correction must be bound to the evidence that fitted it and may not be inferred from a downstream held-out result.

For post-selection CV, a fold-local transform or metric is valid only when the CV owner explicitly records the fold training partition, protected relations, and protocol identity. A fold-local product can change the fold's evaluation realization; it cannot change the frozen target collection or any member's exact membership. For each frozen size, final production uses the accepted method and its complete T_N.

Selection inputs are not a second selector

Representative density, diversity, environment coverage, protected events, difficulty, condition balance, and provenance/correlation structure remain useful scientific information. The current owner represents them as:

```
fitted feature coordinates/metrics
hard obligations or applicability masks
representative-density and diversity evidence
event/environment/condition evidence
difficulty and correlation identities
```

The target-size policy combines these inputs into the one deterministic `pi_train`. There is no competing quota/FPS plan whose prefixes can disagree with that order. A materialization or export record may describe a consumer view of a frozen `T_N`, but it is not an independent membership authority.

Objective, weighting, and exposure

Target membership, target size, loss weighting, and runtime exposure are separate decisions, and the three weighting layers are themselves separate owners applied at different points in the loss:

- `TrainingObjectivePolicy` binds the **global** energy/force/stress coefficients (default `1 : 10 : 1`), head weights, normalization, robust-loss choices, and missing-label behavior. The coefficients are applied exactly once, at the global loss layer, and are emitted explicitly into every generated MACE configuration;
- `ConfigurationWeightPolicy` binds the **per-configuration** weight from applicable condition, regime, event, and quality evidence;
- per-frame **property weights are local availability masks** (`1.0` present, `0.0` absent). They never duplicate the global coefficient ratio, because a per-frame copy would both apply the objective twice and destroy the mask.

The executable loss *family* is not an objective-policy field: it belongs to the canonical MACE method/architecture owner, which is where a loss-family change retires descendant evidence. The executable realization must honour the separation above linearly: the current loss family is MACE's weighted energy+force+stress loss, whose reductions consume the configuration weight and local property weights linearly under the global coefficients. Exposure binds the head, actual gradient exposures, batching/duplication behavior, seed, and runtime lineage.

Optimizer-progress amplitude for the target-size screen is a further separate decision: it is normalized against one configurable reference size so a larger candidate does not also receive more optimizer progress. See Part V.

A frame can be selected once, weighted non-uniformly, and exposed through a qualified loader without those decisions becoming one authority. A custom atomwise or auxiliary loss changes `TrainingProtocolIdentity` and requires its own accepted method identity; it cannot be smuggled into the current protocol through a loader option.

Material and profile specialization

Condition axes and focus groups are declared by the applicable material/profile contract. They may include composition, temperature, pressure, strain, phase, defect, surface/interface state, conformer, preparation history, or another scientifically justified axis. A profile may define hierarchical applicability rather than a Cartesian product. Empty or physically inapplicable combinations are not missing observations merely because their names exist.

Material-specific concepts remain explicit extensions. LTA ring/cage/site groups or Li/Na/K focus groups are not generic defaults and cannot silently change the global target order.

Dependency boundary and failure semantics

The allowed dependency direction is:

```
raw source / label / feature / event evidence
    -> neutral statistical substrate and protected relations
    -> P_train/M3 split and canonical orders
    -> common fitted preparation
    -> optional target-size diagnostic screen and reducer (recommends only)
    -> operator-owned provisional design
    -> frozen design (selected sizes, memberships, role horizons) at cross-validate admission
    -> post-selection fold partitions and method acceptance
    -> fresh final production
    -> downstream qualification roles when separately implemented and activated
```

Forbidden reverse dependencies include held-out CV error choosing target size, locked evidence tuning preparation or checkpoint policy, calibration fitting the protocol it evaluates, and executor/cache behavior changing membership or evidence roles.

The workflow fails closed when labels or protected relations are unresolved, requested roles are infeasible, a fitted product has the wrong lineage, a fold would split an inseparable relation, or a downstream result is offered as selection authority. Explicit absence or deferral is evidence; it is not a synthetic pass.

Part IV - Training, evaluation, and downstream qualification boundary

Purpose and ownership

This chapter defines the training-protocol identity, replay boundary, checkpoint admissibility, post-selection cross-validation, and fresh final production consumed by the current campaign. Target membership and target size are already frozen by Part V; this chapter never creates a second size or membership authority.

Deployment, physical-observable comparison, uncertainty calibration, and locked testing remain product capabilities, but their downstream qualification consumers are outside the P6 public lifecycle. They may consume a current final-production publication only through a separately implemented and explicitly activated successor contract. They cannot feed selection or choose another final model.

Complete training-protocol identity

Multi-head replay fine-tuning trains a shared MACE backbone on target data and an authorized foundation replay corpus with separate output heads. Replay can constrain forgetting while the target head adapts, but replay evidence is not a target-size ranking signal.

Every compared run binds a complete `TrainingProtocolIdentity`, including as applicable:

```
foundation checkpoint / model family / selected foundation head
protocol-global frozen target design and exact membership bindings
replay source, split, and replay-monitor identity
training objective and configuration/property weights
executable loss family
target/replay head weights and realized exposure policy
checkpoint metric and admissibility policy
optimizer, LR schedule, epoch cap, stopping policy, and seed policy
model precision, acceleration backend, and MACE adapter/runtime lock
```

Objective and weighting layers

Three weighting owners are applied at three different layers and are never merged:

- **global loss coefficients** - `[objective]` (`TrainingObjectivePolicy`), default `energy : forces : stress = 1 : 10 : 1`. They are emitted explicitly into every generated MACE configuration, so MACE's own `forces_weight = 100` default never applies to an mdstats run;
- **per-configuration weight** - `[weighting]` (`ConfigurationWeightPolicy`), exported as `config_weight`;
- **local property weights** - availability masks, `1.0` when the canonical label is present and `0.0` when it is absent. They are not per-frame copies of the global ratio.

The executable loss family is MACE's weighted energy+force+stress loss (`loss = "stress"`, `WeightedEnergyForcesStressLoss`), whose native reductions consume `ref.weight` and the local property weights linearly while applying the global coefficients once. MACE's `UniversalLoss` is not used: it scales residuals inside a Huber evaluation - so its per-config property weights are not linearly equivalent to global coefficients - and it does not consume `config_weight`. The loss family is part of method identity, so historical checkpoints trained under different loss semantics are not prefixes or equivalents of corrected trajectories.

Target-size screening, post-selection cross-validation, and fresh final production resolve these owners through the same configuration resolvers, so "the same objective" means the same resolved policy on every path.

The identity contains no unbound caller-held model or fold result. A change to replay semantics, objective, selected membership, checkpoint policy, precision/backend, stopping/LR policy, or another protocol field creates a new method identity and invalidates the descendants that depend on it.

Target and replay evidence

Target and replay retain separate source/label identities, split and exposure accounting, weights, and monitors. Replay preparation never silently acquires an external corpus. True-label replay is compared against its authorized labels; pseudo-label replay, when explicitly supported by the method contract, measures drift from the bound foundation model on an unseen monitor.

`ReplayRetentionPolicy` binds its metric, baseline, permitted degradation, aggregation, and failure semantics. A checkpoint that violates a mandatory replay-retention requirement is inadmissible even when its target metric improves. Replay values receive no target-size ranking, tie-break, fold, or seed credit.

Replay label policy

A campaign declares one external replay source and one canonical label policy. Omitting every label selector next to that source resolves to the source `TRUE_DFT` labels, and generated configuration states that default explicitly. Foundation pseudo-label replay is always an explicit opt-in; it is never a silent fallback for missing or incomplete source labels. Agreeing legacy compatibility spellings normalize to the same canonical choice, while conflicting or unsupported selectors, inexact split domains, mixed replay interfaces, and replay declarations incompatible with the resolved training method are rejected rather than defaulted.

Replay stage ownership

Replay science has exactly one construction owner. `doctor` validates prerequisites - replay topology, source accessibility and label inventory, and, for explicit pseudo mode, the frozen foundation/head/runtime/acceleration realization - and constructs nothing: it runs no replay-wide prediction, creates no mode-specific materialization, and publishes no current replay record. Prediction-dependent eligibility, cardinality, and qualification are reported as deferred.

`prepare` owns construction and reuse: source/index/true-label authentication, the foundation prediction cache and its qualification under explicit pseudo mode, the deterministic split, the training transport and the independent mandatory `TRUE_DFT` monitor, the post-qualification minimum counts, realized replay qualification, and retirement of the prepare-owned accelerator state before it returns. TRUE_DFT preparation performs zero foundation inference. Public `prepare` coordinates two independent preparation owners - target-size and replay - without merging them into one scientific generation, so a replay failure after target-size publication makes public prepare incomplete while the target-size generation stays valid.

Post-selection consumers are scientific readers. Cross-validation, final production, restart/continuation, and representative re-evaluation authenticate the published replay authority; none of them may build foundation predictions, requalify under a changed policy, or create a scientific split. They may reconstruct only disposable representations - a source byte index, a role view, a transport receipt - from parents that are already authenticated, and missing required prediction values route to `prepare`.

Current replay records and external input stability

Current replay records are one exact interface- and mode-appropriate alias set, published atomically after the filesystem products exist. Switching pseudo/true mode, and switching between single-source, no replay, and supported legacy split-file replay, retires the aliases that no longer belong in the same short transaction. Physical content-addressed caches, materialized views, and immutable evidence are left to their own owners and are never

deleted to tidy the mutable current namespace. The long build never holds the campaign-state writer lock, and a stale builder that finishes after a newer `prepare` cannot overwrite the newer current authority.

The replay source is external mutable input. A content check taken before a long streaming operation proves only what the file was then, so every frame consumed during replay construction reproduces its authenticated canonical geometry identity before any dependent prediction or materialized row is recorded under it, and the whole source is re-authenticated once more before prepared aliases become current. A mid-build geometry mutation therefore cannot leave a prediction cache that later validates for the old geometry set. A same-key cold prediction build is single-flight: two concurrent prepares produce exactly one foundation-inference owner, and the internally constructed provider has exactly one terminal retirement owner across success, failure, ownership transfer, and catchable cancellation.

What is not replay scientific identity

Canonically equivalent configuration spelling and execution/storage realization are not scientific identity. An omitted TRUE_DFT mode and an explicit `label_mode = "true_dft"` are one preparation identity. Prediction batch width, physical shard size, graph-cache locator/layout, progress reporting, and a process-local learned OOM-safe batch never trigger foundation reinference and never change replay or post-selection lineage. An identical-byte source relocation is an operational rebind, not a scientific change. A genuine effective-mode, source-content, prediction-policy, qualification, or split change does change lineage.

For explicit pseudo mode, a source mutation that changes only `TRUE_DFT` labels while geometry is unchanged preserves the valid foundation predictions, their qualification, and the split, and refreshes the independent mandatory `TRUE_DFT` monitor on its own. Replay lineage then changes, so the dependent cross-validation and final-production evidence becomes historical.

Monitoring and checkpoint choice

The common target monitor is development/model-selection evidence. It supplies no gradients and is distinct from post-selection held-out fold evidence and from future locked-test evidence. Monitor cardinality is never target-size authority.

`CheckpointMetricPolicy` defines the primary target objective and every mandatory target, focus-group/species, condition, property, replay, and integrity constraint applicable to checkpoint admission. A typical constrained choice is

$$\min_{c} L_{\text{target monitor}}(c)$$

subject to requirements such as

$$L_{F,g}(c) \le \delta_g, \qquad \Delta L_{\text{replay}}(c) \le \delta_{\text{replay}}.$$

Exact thresholds and aggregation are specification-owned serialized policy. Checkpoint choice is deterministic over the complete authorized candidate set and fails closed when no candidate satisfies a mandatory constraint.

MACE adapter and data boundary

The MACE adapter binds package/source identity, head ordering, loader realization, scheduler/stopping behavior, checkpoint retention, precision/backend realization, and any current runtime lock. Documentation URLs are not a runtime contract. Material upstream behavior changes fail closed until the adapter contract is revised and requalified.

Extended XYZ contains only MACE-readable labels, weights, and compact stable identities. Sidecar manifests carry long provenance, policy identities, and audit reasons. Target export includes the declared energy channel,

forces, authorized stress, configuration/property weights, cell/PBC, atom order, and exact label/E0 provenance. Export precision and round-trip behavior are checked through the current reader path.

An `AtomicReferenceFitRecord` becomes the explicit numerical representation accepted by the MACE runtime, normally an atomic-number mapping. A record name or path is not an E0 payload. Target and replay label domains are checked for compatibility rather than silently merged.

The current MACE execution lock extends this boundary through dependency argument mutation. Parser-facing configurations explicitly carry `multiheads_finetuning = false` for ordinary one-head runs and `multiheads_finetuning = true`, `loss = "stress"`, `force_mh_ft_lr = true`, and `real_pt_data_ratio_threshold = 0.0` for replay. The one source-qualified mdstats wrapper prevents pinned MACE 0.3.16 from replacing that loss with `UniversalLoss`, then records the native resolved `WeightedEnergyForcesStressLoss`, LR/EMA settings, replay exposure, source-probe identity, and method/config digests in the existing TRAIN2 runtime evidence. Target-size executions additionally retain the final target batch, realize `ceil(N / B)` batches with complete target UID coverage, and fail closed for an unqualified distributed sampler. This is an execution realization of the existing method identity, not a second trainer or loss owner.

Controlled target-size screen versus ordinary training

The target-size experiment is the special Part V protocol-comparison control. It uses authenticated `n1 -> n2 -> n3` continuation, paired optimizer seeds, direct `M1/M2/M3` endpoint populations, and no ordinary target-success early stopping before a required screen boundary. An earlier checkpoint cannot replace the prescribed endpoint merely because its metric is better.

The current public screen owns the complete restartable continuation. Generated campaigns default to `(n1,n2,n3) = (1,3,10)`; fresh final production has its own independent role horizon `H_prod_i`, frozen per selected size when the downstream design is frozen rather than read from `[training].max_num_epochs` at execution time. Screen checkpoints and CV checkpoints are never production parents.

After selection, CV and final production run under the accepted method. CV uses fold partitions of exactly `T_selected`, with fresh model/optimizer lineage per required fold/seed. Final production starts fresh from the accepted foundation and trains the complete `T_selected`; it continues no screen or fold trajectory. Its run namespace remains disjoint even when a numeric seed or target size coincides.

Post-selection method acceptance

Post-selection work runs once per frozen selected size, in frozen selection order, over one shared prepared generation and one shared method resolution. The per-size dependency graph is acyclic:

```
per-size target binding                        (N_i, exact T_i, prepared/P2 lineage)
  -> shared post-selection method identity
  -> CV policy (contains H_cv_i) and final-production policy (contains H_prod_i)
  -> CV plan and final-production plan
  -> fold/final execution and evidence
```

The target binding names the target and nothing else: no role horizon, no selection provenance, no sibling size, no list position, and no digest of a record carrying them. That is what lets the production budget be edited without invalidating accepted cross-validation evidence, lets a size chosen by hand and the same size adopted from the diagnostic be one experiment, and lets a second selected size join the design without disturbing the first size's evidence.

The shared method identity binds preparation/objective recipe, executable loss family, foundation and initialization family, optimizer family, LR schedule, checkpoint semantics, precision, and backend. It does not contain fold membership or a second target size.

One canonical optimizer-setting resolution

The shared scientific optimizer semantics a campaign may configure - general learning rate, batch size, validation batch size, evaluation interval, EMA enable/disable, EMA decay, AMSGrad, weight decay, gradient clipping, and the permitted optimizer family - are resolved exactly once, by `mdstats.training_data.training_settings.resolve_shared_optimizer_settings`. The learned-model dtype is likewise resolved once, by the binary learned-model-precision contract in the same module. The method identity, the executable `MaceOptimizerPolicy`, the generated MACE configuration, and the TRAIN2 runtime all descend from those single resolutions:

```
configuration
  -> one canonical resolved value
      -> P5 method identity
      -> executable MaceOptimizerPolicy
      -> generated MACE config
      -> TRAIN2 runtime
```

There is no second, independently defaulted route. P5 method identity is therefore literally the method that executes: an explicitly configured shared optimizer field changes both the recorded identity and actual training, and an omitted field resolves to the same default on both sides. The optimizer seed, role-specific epoch budgets, and worker counts stay outside these shared settings, because they are per-run identity, role policy, and pure resource choice respectively.

The resolver is also the place where the configuration domain is enforced, so that a malformed setting cannot become either a recorded identity or an executed method: learning rate, EMA decay, weight decay, and gradient clipping must be finite reals in their declared ranges; batch sizes and evaluation interval must be exact positive integers, never booleans or truncated floats; and the EMA and AMSGrad flags must be actual booleans rather than truth-normalized values. `MaceOptimizerPolicy` repeats those invariants in its constructor, because it is independently constructible and independently deserialized.

The same fail-closed, validate-before-canonicalization rule applies to the target-size normalization reference policy, `TrainingObjectivePolicy`, and `ConfigurationWeightPolicy`, including their current-schema readers. Real values must be finite and in their declared ranges, integer values must be actual integers, boolean values must be actual booleans, and collection elements must satisfy their declared domains. A malformed current-schema value is rejected before it can affect identity, export, or execution; only an explicitly supported historical reader may preserve a historical representation.

Because historical evidence could previously record an identity whose defaults were never the ones execution applied, the method recipe carries an explicit version. Evidence produced under the earlier resolution cannot authorize corrected cross-validation or final production; it remains readable as history and is never rewritten in place.

The CV policy owns `K >= 2`, partition seed, fold algorithm, CV budget, monitor/purge allocation, target-only acceptance, and the all-required-fold / all-required-seed rule. The final-production policy owns the production epoch horizon, production seed matrix, and committee policy. Neither policy can rewrite the other or the selected binding.

The CV plan records the current selected binding, protected P1 relations, selected-only fold memberships, and required run matrix. The final-production plan records the complete `T_selected` and accepted CV authorization. Evidence descends from a plan and binds it; corrupted evidence invalidates itself and never rewrites its authorizing plan.

CV freezes each fold representative on its authorized target monitor before evaluating the held-out fold. A required fold or seed failure is a methodological failure: it leaves the frozen design and its evidence unchanged and does not authorize final production.

A materially different *target* method or population requires a new target-size experiment, because the quantity the screen measured has changed. The current target-size screen is target-only and carries no replay exposure, so this is not a rule that every downstream method change regenerates the target-size experiment: a replay-only method or lineage change leaves the prepared generation, the screen evidence, the provisional selection, and the frozen collection intact, and instead makes the post-selection descendants that measured the old replay lineage historical, requiring new cross-validation under the same frozen target collection.

Public status reflects that separation. A binding-keyed post-selection or qualification pointer survives a replay-only change because the target binding is unchanged, so pointer existence is not currentness: current status checks that the pointed descendants still bind the replay lineage that is current now. The target revision, the compact current replay lineage, and every P5/P7 pointer row are read in one coherent observation snapshot, so a concurrent publication cannot produce a status combination that never existed. Nothing is deleted to make status correct.

Campaign-level acceptance is all-sizes. Cross-validation is accepted only when every frozen size is accepted, and final production admits no new run for any size until every frozen size holds current accepted cross-validation ancestry under its own binding. A failing size stays visibly failed and is never dropped; valid completed sibling evidence stays reusable on retry.

Final production and currentness

Final production publishes only after reauthenticating the current campaign revision, selected binding, accepted method, and complete production plan. `ProtocolFreezeRecord` binds the method, selected membership, replay/monitor identities, checkpoint/committee identities, and upstream evidence needed by the current production consumer.

Every current read resolves the selected binding again from the store; it does not trust a stale caller object. Publication rechecks currentness in the same transaction that would make a descendant current. A superseded run can retain diagnostics, but it cannot publish a current final model.

Downstream product boundary

Physical observables such as RDF, coordination, topology, MSD, VACF, spectra, VDOS, diffusion, and conductivity remain owned by their analysis modules and their own specifications. A future downstream qualification recipe must bind matched reference/candidate collection identity, runtime/capability identity, analysis-owned result identity, and an explicit statistical role.

Calibration is valid only for predictions from the actual frozen final committee. Locked-test evidence remains sealed until its explicit activation boundary. Neither calibration nor locked evidence may alter fitting, target membership, target size, training protocol, checkpoint selection, or final publication. P6 does not claim that these downstream consumers are implemented or qualified.

Failure and reproducibility semantics

The workflow fails closed for incompatible label domains, missing foundation or replay identity, unsupported loader exposure, missing required fold/seed, stale selected binding, invalid checkpoint constraints, corrupt checkpoint state, or a downstream result offered as selection authority.

Reproducibility binds source/label and protected-role identities, the neutral substrate, target-size experiment and orders, common preparation, selected binding, method/policy/plan identities, replay/monitor identities, optimizer/LR/stopping/seed policy, precision/backend, checkpoint evidence, and published final identity. Worker

count, queue order, cache path, and other execution-only choices remain outside scientific identity unless a current specification explicitly says otherwise.

Part V - Target-size selection and post-selection validation

Purpose and ownership

This chapter defines how the campaign decides **how much labelled data a training method needs**, and how that decision is validated afterwards without contaminating it.

The current chain is:

```
canonical frame authority (Part II)
    -> neutral statistical substrate (Part III)
    -> one P_train / M3 target-size development split
    -> one canonical training order pi_train
    -> one canonical evaluation order pi_eval with nested M1 subset M2 subset M3
    -> one common deterministic target-size preparation
    -> +-- optional: paired optimizer-seed automatic diagnostic
    |   |             (screen + reducer -> a *recommended* size)
    |   |
    |   +-> operator-owned provisional design
    |         N_provisional, H_cv, H_prod   (mutable, freezes nothing)
    -> cross-validate admission
         -> frozen design: every selected size N_selected, its exact T_selected =
pi_train[:N_selected],
            and both effective role horizons
    -> post-selection cross-validation on the frozen collection
    -> fresh final production on the selected dataset(s)
    -> currentness-fenced publication
```

Each element has exactly one owner. The **operator** decides the target size and the two role horizons; the automatic screen's reducer produces evidence and, when its comparison is valid, a *recommendation*; **`cross-validate` admission** is the only authority that freezes a downstream design; `CampaignStore` is the only authority that holds the current provisional or frozen design; post-selection cross-validation is the only authority that accepts or rejects the training *method*; and final production is the only authority that publishes a production model.

Why the screen recommends rather than decides

The screen measures target-force RMSE at three short epoch boundaries under one configured protocol. Fixed-optimizer and normalized-optimizer experiments on this campaign showed that candidate size and short-horizon optimizer progress are strongly coupled, and that reasonable optimizer controls can materially change the ranking across sizes. That evidence does not establish which control was closer to long-horizon truth, that the size/error relationship is linear, that an early ranking predicts a long training trajectory, or that force RMSE predicts the stability of the resulting potential in molecular dynamics.

So the product does not let a short-horizon proxy masquerade as a conclusive target-size authority. The screen is retained because its empirical content is genuinely useful; its *scientific interpretation* is narrowed to what it measured. Evidence strength increases monotonically downstream:

```
automatic screening   -> target-size heuristic / diagnostic evidence
post-selection CV     -> validation of the training method on the chosen data
full production       -> long-horizon realization of the chosen design
MD qualification      -> evidence about actual potential behavior
```

No target-size screening metric is evidence of final MD quality.

There is no alternate selection path. The retired per-domain multi-view chain (compatibility-domain role freezes, full-pool feasibility, exact sparse neighborhood indices, progressive multi-view ordering, repaired master orders, continuation-state families, and independent prefix qualification) is not a current architecture, is not migrated, and is not reachable from any current runtime owner. Workspaces holding that derived state are rejected with an actionable destructive-reset requirement rather than translated; see Part VII.

Why target size is decided by a screen, not by coverage

The scientific question is empirical: *at what training-set cardinality does the accepted training method stop improving materially on a representative held-in evaluation population?* That is a property of the method, the data distribution, and the optimizer - not of a geometric covering argument over descriptor neighborhoods.

Four principles control the design:

1. one deterministic training order, so candidate sizes are exact nested prefixes and size comparisons are never confounded by resampling;
2. one common preparation shared by every candidate size and optimizer seed, so preparation cannot become a hidden per-size variable;
3. only the ordered optimizer-seed set is the stochastic replicate dimension of the screen;
4. the decision consumes target-side metric evidence alone, and downstream replay, cross-validation, physical, or deployment evidence can never rank or tie-break a size.

The development split and the two canonical orders

The neutral statistical substrate supplies protected relations - duplicate groups, correlation families, and split exclusions - before any target-size object exists. From it the architecture derives exactly one split:

```
eligible labelled frames -> P_train (target-training pool) + M3 (evaluation pool)
```

`P_train` is ordered once into `pi_train`. A candidate of nominal size `N` is the exact prefix

$$T_N = \pi_{\text{train}}[:N].$$

`M3` is ordered once into `pi_eval`, and the evaluation ladder is the nested family

$$M_1 \subset M_2 \subset M_3,$$

taken as direct prefixes of `pi_eval`. Rungs are direct populations, never complements of one another: a rung is evaluated on exactly the frames it names.

Both orders are deterministic functions of the canonical frame authority, the neutral substrate, and the configured target-size policy. Neither depends on any compatibility grouping, label-domain identity, or cross-validation plan.

The common preparation

One `TargetSizeCommonPreparation` identity is frozen before any candidate trajectory starts. It derives from `P_train` and the configured foundation/training protocol, and it is shared unchanged by every candidate size and every optimizer seed.

It MUST NOT derive from `M1`, `M2`, `M3`, held-out evidence, calibration evidence, locked evidence, or any cross-validation plan. A change to the common preparation is a change to the target-size scientific identity and produces a new generation rather than an in-place edit.

The paired optimizer-seed screen

For every candidate size `N`, the screen runs the same ordered optimizer-seed set - by current policy the two seeds `[1, 2]` - through the same fidelity ladder:

```
n1 / M1  ->  n2 / M2  ->  n3 / M3
```

Fidelity boundaries are continuation points, not restarts: model, optimizer, and RNG state continue exactly across `n1 -> n2 -> n3`. Ordinary early stopping may not truncate a required screen boundary, and the seed set is identical at every `N` so a size comparison is never a seed comparison.

Optimizer-progress normalization

Under a fixed number of dataset passes and a fixed batch size `B`, candidate `N` performs `U_N = ceil(N/B)` optimizer updates per epoch. Holding the nominal learning rate and EMA decay fixed would therefore give larger candidates strictly more optimizer progress - a second independent variable the target-size question never asked about. The screen removes that confound by normalizing amplitude against one configurable reference size:

```
U_ref   = ceil(N_ref / B)
U_N     = ceil(N / B)
s_N     = U_ref / U_N
lr(N)   = reference_learning_rate * s_N
beta(N) = reference_ema_decay ** s_N
```

so `lr(N) * U_N` and `beta(N) ** U_N` are invariant in `N`. An exact doubling of update geometry halves the learning rate and takes the square root of the EMA decay. Defaults are `N_ref = 1024`, `reference_learning_rate = 1.0e-4`, `reference_ema_decay = 0.99999`, configured under `[target_data.size_convergence.optimizer_normalization]`. The reference size need not be a candidate and need not lie inside the configured ladder. No cap, floor, clipping, survivor-dependent rescaling, or candidate-specific override is applied.

EMA is normalized because EVAL2 evaluates the authenticated configured model state, which is the EMA state whenever EMA is enabled; leaving the decay fixed would compare EMA windows of different effective lengths.

The normalized clock is also the executable loader contract. The qualified target-size MACE path retains the final partial target batch, uses no truncating target sampler, and realizes exactly `ceil(N / B)` target batches. Every exported target `frame_uid` must occur once in that epoch; target frames are never duplicated merely to fill a batch. A distributed target-size path that cannot prove the same coverage fails closed rather than silently changing the frozen `ceil` geometry to floor semantics. Resolved loss, optimizer, replay, batch, membership, and source-probe facts are recorded in the existing runtime evidence and are required for continuation/currentness.

Only those two update clocks are normalized. Epoch and fidelity boundaries, the number of dataset passes, the batch size, the LR phase fractions and normalized-progress multiplier shape, Adam/AMSGrad settings, weight decay, gradient clipping, model precision and architecture, acceleration policy, the optimizer-seed set, and the objective/weighting policy are all held fixed across candidates. This is a first-order optimizer-progress normalization, not a claim of exact optimizer-path equivalence: minibatch noise, Adam moment history, and the finite discretization of the analytic LR curve remain accepted residuals.

Each `(N, optimizer_seed)` derives its scale, effective learning rate, and effective EMA decay **once**, from the full candidate geometry, and binds them into the candidate realization. The same realized values are replayed through every rung of `n1 -> n2 -> n3`; they are never recomputed from the active rung or the survivor set, so eliminating a candidate cannot alter a surviving candidate's schedule. Restart validation re-derives the normalization identity and rejects drift, so a stale fixed-LR or differently-normalized checkpoint cannot be resumed.

This normalization is a control of the size-comparison **screen** only. Post-selection cross-validation and fresh final production start from their own optimizer state under their own accepted method policy; screen checkpoints are never production parents.

The normalization policy is the screen's *only* learning-rate and EMA-decay authority. General `[training].learning_rate` and `[training].ema_decay` are post-selection/general training settings; they never

reach screen training and are not part of target-size scientific identity. Editing them cannot retire an otherwise identical screen.

The normalization resolver and current-schema reader enforce the reference domain before digesting or projecting a candidate: `reference_target_size` is a positive integer, `reference_learning_rate` is a finite real greater than zero, `reference_ema_decay` is a finite real strictly between zero and one, and the algorithm identifier is the specification-owned string. Strings, booleans, fractional values, and non-finite values are rejected rather than coerced.

What is, and is not, target-size scientific identity

The target-size screen projects the generic optimizer carrier down to the fields that actually change a candidate trajectory. That projection is built once and is used identically by fresh screen construction, restart-authority construction, active-candidate resume validation, and terminal/currentness reconstruction, so those four paths cannot disagree about what the screen's method is.

Target-size scientific identity **retains** the training `batch_size` (which fixes the `ceil(N/B)` update geometry), EMA enabled/disabled, AMSGrad, weight decay, gradient clipping, learned-model dtype and critical precision, device/acceleration policy, and the full-screen `n3` horizon taken from the screen schedule.

It **excludes** the optimizer seed and the candidate-local acceleration realization (both rebound per candidate), general `[training].learning_rate` and `[training].ema_decay` (replaced by the normalization policy), `num_workers` (pure resource realization), the harness-validation `valid_batch_size` (fixed, non-controlling validation geometry that moves no gradient trajectory, LR schedule, checkpoint admissibility, or ranking), and `eval_interval` (not emitted into the candidate MACE configuration at all).

General `[training].ema_decay` is inert for the screen in both EMA states, and the candidate configuration reflects that. With EMA enabled the configuration carries the size-normalized realized beta, which is strongly authenticated. With EMA disabled there is no EMA state to decay, so no decay key is emitted at all - writing the generic value there would put an inert number into scientific replay and let a post-selection-only edit reject an accepted trajectory. Toggling EMA on or off is, by contrast, genuine screen science, because EVAL2 consumes the EMA state whenever EMA is enabled.

Before any of this is resolved, the shared optimizer configuration domain is checked exactly: non-finite learning rates or decays, fractional or boolean batch sizes, and non-boolean EMA/AMSGrad flags are rejected at the canonical owner, before they can reach method identity, a candidate trajectory, a materialization, or a trainer launch.

Consequently a mid-screen worker-count or harness validation batch-width change is not scientific invalidation: a published candidate materialization keeps the exact execution values it was launched with as historical execution provenance, restart re-derives and compares only the scientific configuration content, and `n1 -> n2 -> n3` continuation ancestry stays exact. Real method drift - batch size, dtype, EMA enable, AMSGrad, weight decay, gradient clipping, or the normalization reference LR/EMA - still rejects before training resumes.

Training batch size and the learned-model dtype are P3 *execution* identity, not common-preparation identity. The common preparation fits atomic references, configuration weights, and the fixed harness membership; it consumes neither value, so a batch or precision edit must not retire prepared P1/P2/common science, while it does still retire the P3 execution descendants whose trajectory it changes.

Objective and weighting ownership

Three weighting owners are kept distinct and are applied at different layers:

- `[objective]` (`TrainingObjectivePolicy`) owns the **global** loss-component coefficients, by default `energy : forces : stress = 1 : 10 : 1`. They are emitted explicitly into every generated MACE configuration - target-

size candidate, post-selection CV, and fresh final production - so MACE's own `forces_weight = 100` default is never in effect. It does not own the executable loss *family*, which belongs to the canonical MACE method/architecture owner;
- `[weighting]` (`ConfigurationWeightPolicy`) owns the **per-configuration** weight, exported as `config_weight`;
- per-frame property weights are **local availability masks**: `1.0` when the canonical label is present, `0.0` when it is absent. They are never per-frame copies of the global ratio.

Target-size common preparation and post-selection resolve all three through the same config resolvers, so a user override cannot be honoured by one path and silently ignored by the other. Changing `[objective]` changes common/prepared-generation identity and invalidates its descendants; changing the optimizer-normalization reference values does not, because normalization is P3 execution identity rather than a preparation input.

The executable loss family is MACE's weighted energy+force+stress loss (`loss = "stress"`, `WeightedEnergyForcesStressLoss`), whose native reductions consume `ref.weight` and the local property weights linearly while applying the global coefficients once. MACE's `UniversalLoss` is not used: it scales residuals inside a Huber evaluation, so its per-config property weights are not linearly equivalent to global objective coefficients, and it does not consume `config_weight` at all. The loss family is part of method identity - a checkpoint trained under a different family is not a prefix of a corrected trajectory.

Candidate rungs execute through the accepted TRAIN2 runtime and are evaluated through the accepted EVAL2 owners. Expensive numerical training has exactly one substitution seam, strictly below the mdstats owner boundary; configuration resolution, authority construction, materialization, provider and checkpoint authentication, publication, reconciliation, and adoption are production code in every invocation.

An unaccepted first-rung materialization is execution-local scratch. Cleanup and first-rung execution for a logical `(N, optimizer_seed)` cell acquire one exclusive execution-local fence, then recheck authenticated progress before reclaiming scratch. A concurrent writer therefore cannot have its live workspace deleted or launch duplicate training; once accepted progress exists, the waiter reuses that immutable evidence. The fence is not scientific state and releases when its process exits, so a later retry may reclaim stale scratch from an interrupted attempt without changing the campaign identity.

The reducer and the diagnostic outcome

One reducer consumes the screen evidence and advances the diagnostic. Its outcome is one of:

- **recommendation** - a size `N` together with the exact membership identity of `pi_train[:N]`;
- **typed scientific no-recommendation** - too few candidates qualified, or the surviving candidates were not comparable.

Neither outcome freezes anything, and neither is campaign-terminal. A diagnostic that cannot make its configured comparison is a conclusion about *the diagnostic*; it leaves any existing provisional choice untouched and does not prohibit choosing a qualified candidate explicitly. Its evidence is persisted, rendered, and reused by a later `--auto`.

Ranking is owned by the target-side metric and practical-equivalence policy alone. Inside the practical-equivalence band the **smaller** `N` is preferred, because the scientific question is the smallest sufficient training-set size.

The configured ladder ceiling is a **practical budget limit**, not a requirement that convergence occur below it. The terminal decision therefore distinguishes two selected outcomes:

- **evidence-supported truncation** - a smaller size is practically equivalent to, or better than, the larger finalist, so there is direct evidence to stop below the ceiling. This is an ordinary selection with no warning;
- **practical-ceiling recommendation** - the largest configured candidate remains materially superior to every other successful terminal finalist by more than the practical-equivalence threshold. `Nmax` is then recommended, and the result carries the non-blocking warning code `nonconverged_at_configured_ceiling`: the configured

practical ceiling is the best evaluated permitted size, while target-size convergence was not demonstrated within the configured ladder. It does not claim that `Nmax` is asymptotically converged, and no unconfigured rescue size is invented.

The warning is diagnostic metadata on a valid recommendation, carried in `terminal_reason_codes`. There is no separate recommended-with-warning status: a ceiling recommendation commits through the ordinary diagnostic-completion transition and, if it is adopted, becomes the provisional `N` like any other. CLI status, the derived result view, and the portable report surface the warning alongside the recommendation.

Genuinely insufficient comparison stays blocking. Too few complete comparable terminal candidates, malformed/missing/duplicated/reordered/lineage-incompatible boundary evidence, and authenticated numerical failures that leave the reducer unable to make the required comparison remain typed failures; the reducer never fabricates a ceiling recommendation from an incomplete terminal comparison.

The terminal-decision rule participates in P2 policy identity (`practical_equivalence_then_practical_ceiling.v2`). Evidence reduced under the retired blocking-ceiling rule stays historical and is never relabelled in place.

The provisional design, and the freeze

The prepared generation is expensive and deliberately reusable, so the operator may want several longer-horizon experiments from it - the same method and the same training order at different amounts of target data. The provisional design is therefore an **ordered collection of distinct qualified sizes**, not a single choice:

```
D_provisional = [ entry_1, entry_2, ..., entry_k ]     unique by N, first-insertion order

entry_i:
  N_provisional      a configured qualified candidate size
  T_provisional      never stored; always pi_train[:N_provisional], with its digest
  selection_source   manual | auto_recommendation   (provenance, not a variable)
  H_cv               cross-validation max epochs for this size
  H_prod             final-production max epochs for this size
```

`select-target-size <N>` merges one complete entry through the ordinary CampaignStore compare-and-set boundary: a size not yet in the design is **appended**, and a size already in it has its **complete entry replaced in place**, keeping its list position. A second distinct `N` therefore adds an experiment rather than erasing the one already requested. Two entries for one `N` in authoritative state are corruption, not something to deduplicate. The empty collection is the canonical unselected state - there is no `N = undefined` placeholder - and `select-target-size --reset` returns to it pre-freeze without touching the prepared generation or any valid diagnostic evidence.

Each entry is *complete when it is set*: both horizons are resolved to explicit values at that moment from `[post_selection.cv].max_num_epochs` and `[training].max_num_epochs`, or from the invocation's own `--horizon-cv` / `--horizon` overrides. Later edits to `campaign.toml` therefore cannot silently mutate an entry that already exists, untouched sibling entries never drift, and a CLI override never becomes a sticky default. Reselecting a size deliberately re-resolves every omitted horizon for the new invocation. The CLI never writes `campaign.toml`.

An adopted automatic recommendation goes through that same merge owner. The automatic path has no collection authority of its own: it cannot clear, reorder, or overwrite a sibling entry the operator chose by hand.

`cross-validate` admission is the freeze, and it freezes the **whole collection atomically**. It reloads and authenticates the one prepared generation, revalidates every proposed `N` against the current qualified candidate set, rederives every `T_N = pi_train[:N]` from the P2 training order, and publishes the complete ordered design - every `N_selected`, its exact membership, and both effective role horizons - as immutable ancestry in one transition, before any numerical CV work. One unauthenticatable member rejects the entire admission; there is no partial

freeze and no quiet reduction to the subset that still validated. No automatic-screen execution head or reducer participates: a campaign that never ran the diagnostic freezes by exactly the same path as one that did. After the freeze, `select-target-size` in any form refuses to change the design.

The full frozen entry, and the target binding

The operator chooses `(N, H_cv, H_prod)` together, but identity does not collapse them - and the separation has to survive being written down, not just being intended.

The **full frozen entry** is the immutable design and audit record: `N`, exact membership and training-order identity, both role horizons, and selection provenance. It may carry its own digest for state authentication, and that digest is what detects a tampered horizon or forged provenance.

The **target binding** is its role-neutral scientific projection, and it is what every P5/P7 descendant descends from:

```
TargetBinding_i   = generation + accepted P1/P2 prepared lineage + N_i + exact T_i + training-
order identity
CV_i              = TargetBinding_i + method identity + CV policy (which contains H_cv_i) + CV
plan/evidence ancestry
Production_i      = TargetBinding_i + method identity + accepted CV_i ancestry + production
policy (which contains H_prod_i)
```

So editing the production budget cannot invalidate accepted cross-validation evidence; editing the CV budget does not move the production policy itself, and reaches final production only through the accepted CV ancestry it names, which is correct. Selection provenance is excluded from every numerical identity: the same `N` on the same substrate is the same experiment whether it was chosen by hand or adopted from the screen. Sibling sizes, list position, and any whole-collection digest are excluded too - a per-size identity that moved when another size joined the design would make the multi-size feature self-defeating.

The binding must therefore not be derived from the full frozen entry's digest, because that digest contains exactly the fields the binding is defined to exclude. This was a real defect in the scalar-selection predecessor, whose binding embedded the whole frozen record and so let `H_prod` and provenance contaminate every descendant transitively. Descendants published under that predecessor schema keep their own bytes and stay current under their own exact ancestry; new designs use the corrected decomposition, and no compatibility argument reintroduces the coupling.

Currentness

`CampaignStore` holds one canonical target-size generation. Its durable regimes are `legacy`, `transitioning`, and `current`; only `current` executes target-size work. Every mutation is one compare-and-set transition against the exact predecessor revision, so an interrupted operation is owned by the persisted transition rather than by the process that began it.

The diagnostic projection binds the recommended `N` and the exact membership digest it names together; neither may be edited independently, and a reload re-derives the projection from the authenticated reducer state and training order rather than trusting the stored copy. Diagnostic currentness is always established from the current store revision, never from a caller-supplied snapshot, and a public diagnostic view is re-authenticated at exposure time so a stale object cannot be published after the store advances. The same holds for a frozen selection: its membership is re-derived from the P2 training order on every exposure.

An automatic diagnostic may run for hours. It captures the selection state it intends to update before it starts, and installs its recommendation only if that state is unchanged when it finishes. If a human made an explicit choice, or `cross-validate` froze the design, in the meantime, the newer decision wins: the diagnostic evidence and its report are still committed and reusable, and the CLI says the recommendation was computed but not installed.

The portable diagnostic report

Every completed automatic diagnostic - with or without a recommendation - writes one self-contained Markdown report under the campaign `results/` tree at a stable per-generation filename. It records the generation and experiment/execution/head/reducer identities; the candidate sizes, evaluation populations, fidelity boundaries and optimizer seeds; the ranking metric and unit, the seed aggregation, the practical-equivalence rule and the funnel rule; the optimizer-normalization reference policy and each candidate's updates per epoch, effective learning rate and effective EMA decay; every completed boundary's per-seed RMSE, paired mean, success/failure and survive/eliminate outcome; the filtering decision tree projected from the reducer's committed outcome history; the recommendation or explicit no-recommendation result with any warning codes; and the scientific-limitation statement above.

The report is derived, rebuildable and non-authoritative. It never re-ranks anything, and no code reads a recommendation or a selected size back out of it.

Invalidation scope

A change to target-size scientific identity - source or frame membership, canonical numerical labels or their interpretation policy, the candidate ladder or configured ceiling, the evaluation-size ladder, fidelity boundaries, the ordered optimizer-seed set, the training-order policy, the `P_train`/`M3` split or `pi_eval` ordering policy, the common preparation, the metric/practical-equivalence policy, the terminal-decision policy, the training objective and its weighting ownership, or the foundation/replay identity where it is part of the experiment - replaces the generation. The old selected set stays readable as history and can never re-enter current authority.

Changes that are *not* target-size identity invalidate only their own descendants:

- advisory provenance grouping or report presentation invalidates only the advisory evidence that depends on it, and never the frame UID, the canonical label identity, the neutral partition, or the target-size result;
- cross-validation-only settings such as fold count and partition seed invalidate cross-validation and its descendants, and leave every `N_selected`/`T_N` byte-identical;
- adding, revising, or removing one size before the freeze changes only that entry; after the freeze the design is immutable, and a sibling size never participates in another size's numerical identity;
- neither provisional nor frozen role horizon participates in automatic target-size diagnostic identity, so steering them never invalidates screen evidence;
- production-only budget or runtime policy invalidates only final-production descendants.

Post-selection cross-validation

Cross-validation performs the whole-collection freeze at its own admission boundary and then runs the existing methodology once per frozen size, in frozen selection order, each consuming exactly its own `T_N` - complete coverage, no unselected sibling frame, no held-out outer frame, and no frame borrowed from another selected size.

The size dimension sits *outside* everything below it: fold construction, seeds, evaluation, and the acceptance predicate are unchanged, and there is no cross-size reducer. Outer iteration over sizes is serial and shares the one effective resource allocation the existing fold/seed/MACE/library concurrency already owns; no new scheduler exists and no size claims the machine independently. Every frozen size receives its own valid cross-validation verdict before any campaign-level reduction happens: a methodological rejection of one size is recorded and the remaining frozen sizes are still cross-validated, because the frozen collection *is* the experiment the operator requested. Campaign cross-validation is accepted only when **every** frozen size is accepted; a rejected size stays visibly rejected and no selected size is ever silently dropped. A hard execution, corruption, lineage, or authority failure is not a scientific verdict and may still abort the invocation.

It validates the **training method**, not the size:

- the configured fold count `K >= 2` and every required fold of every required CV seed must pass the configured target-only acceptance predicate; there is no mean, majority, best-seed, partial, `K = 0` or `K = 1` authorization;
- the full P1 split-exclusion and correlation-family constraints continue to hold inside fold assignment;
- fold-local preparation, training, checkpoint selection, and replay admissibility may never see that fold's held-out outer target set, and the fold representative freezes before held-out outer evaluation;
- replay training exposure and the TRUE_DFT replay admissibility monitor remain distinct concerns, and TRUE_DFT replay contributes no ranking, tie-break, fold, or seed credit;
- a cross-validation failure is a methodological result: the frozen design and its evidence are unchanged, and final production is simply not authorized for any size. If cross-validation shows that a materially different training method is required, that changed method needs a **new** target-size experiment, because the method whose convergence was measured has changed;
- valid completed sibling evidence stays reusable on retry under the existing currentness and restart rules.

Supported training modes remain exactly `scratch`, `naive_fine_tuning`, and `multihead_replay`; the canonical post-selection heads remain `target_head` and `pt_head`; and the foundation checkpoint head remains a separate foundation-owned concept. Method, foundation, replay, and content identity all fail closed.

Foundation **identity** is the authenticated checkpoint content, selected head, and family; the configured filesystem path is a runtime locator only. One canonical interpretation resolves that locator for every downstream owner - ~ expands to the user home, a relative value is anchored to the campaign configuration directory rather than the process working directory - so one `campaign.toml` cannot mean different files to environment checks, method identity, execution, and qualification. Relocating byte-identical content with the same selected head therefore preserves the method and stays executable after reauthentication, while changed bytes, head, or family still invalidate stale descendants fail-closed.

Fresh final production

Admission is a **collection-wide barrier**. Before any new production job starts, the complete frozen design is authenticated and every selected size must hold current accepted cross-validation ancestry under its own binding and its own `H_cv`. If any size is missing, stale, corrupt, incomplete, or rejected there, the invocation starts no production job for *any* size and names every known blocker. Immutable evidence from an earlier attempt keeps whatever currentness its own identity earns; the barrier exists so that an all-sizes experiment cannot quietly become the subset that happened to succeed.

After preflight, each frozen size starts fresh from the accepted foundation/initialization with fresh optimizer, RNG, and run state. It trains on that size's complete exact `T_N`, under the cross-validation-accepted method for *that* size, for **its own** frozen production horizon - an independent budget that is deliberately unrelated to the screen's `n3` and to the frozen CV horizon. Each size publishes one binding-scoped final-production decision. No size may consume another size's membership, horizons, CV plan or acceptance, run evidence, pointer, final plan, or publication, and there is no cross-size final-publication committee.

Frozen `M3` evidence may remain development/model-selection evidence. Final authorization and publication remain currentness-fenced and restart-authenticatable: a reopened campaign reauthenticates each selected binding, its cross-validation acceptance, and its final publication identity before exposing any of them as current. If one size is complete and another fails or is interrupted, the frozen design is unchanged, the complete size's evidence stays reusable, only work the existing restart owners deem incomplete or stale is recomputed, and campaign production stays incomplete until every size closes.

Where the multi-size experiment ends

Once every selected size has a current final publication, a multi-size training experiment is **complete and not release-qualified**. Several final products exist and this revision authorizes no rule for choosing one, so

`advance` offers no further consequential command, `qualification status` reports the boundary read-only, and consequential qualification commands fail closed before any attempt or locked evidence is created or revealed. A one-size design keeps the existing qualification path unchanged. Choosing between sizes is a release decision that needs its own explicit design authority; it is deliberately not made here by defaulting to the first, the last, the recommended, or the best-scoring size.

Public command surface

The current lifecycle, including configuration initialization, is exactly:

```
init -> doctor -> prepare -> select-target-size -> cross-validate -> train-production
```

`prepare` reconstructs the current substrate and cannot select a size. `select-target-size <N>` sets the provisional design and trains nothing; `select-target-size --auto` runs or reuses the optional automatic diagnostic and adopts its recommendation; a bare `select-target-size` is invalid, and `<N>` and `--auto` are mutually exclusive. `cross-validate` freezes the design and is the only command that accepts the method. `train-production` is the only command that publishes a fresh production model. `status` and `advance` project this lifecycle from the owning authorities rather than from stage markers, and `advance` stops at the target-size decision boundary rather than inventing a choice or silently running the diagnostic.

Part VI - Bounded execution, restart, and performance architecture

Purpose and authority

Execution optimization is acceptable only when it preserves the scientific and statistical authorities in Parts I-V and improves measured throughput, memory, storage, or restart cost. Utilization is diagnostic; authenticated records, deterministic decisions, and exact scientific digests decide correctness.

Worker count, queue depth, query-block size, cache location, file-backing threshold, storage path, and similar execution choices do not enter scientific identity unless a current specification explicitly makes them part of the algorithm.

The central rule is:

> change how exact work is scheduled or represented, not what evidence is consumed or what authoritative decision is produced.

Work/span and single-level parallelism

For serial work (T_1), critical path (T_), and (P) admitted CPU lanes,

$$T_P \geq \max\left(\frac{T_1}{P}, T_\infty\right).$$

Independent work is exposed at the highest useful level. Nested numerical parallelism is suppressed while outer work fills the resource budget:

$$P_{\text{outer}} P_{\text{native}} \leq P_{\text{budget}}.$$

The resource scope controls cKDTree, BLAS, OpenMP, PyTorch, and other native threads. A process or worker does not independently oversubscribe the host. The implementation may use exact kernels such as `query_ball_point`, `numpy.bincount`, bounded indexed reductions, or `threadpoolctl`; these are execution realizations and do not change evidence roles or canonical order.

Preparation and execution boundaries

The source/frame and numerical-label authorities are built once and validated through their current owners. The neutral statistical substrate supplies the one `P_train/M3` split and the two canonical orders. The current P3 common preparation is then computed once and shared by all authorized candidate sizes and optimizer seeds.

```
source/frame/label authorities
  -> neutral statistical substrate and protected relations
  -> one P_train/M3 split and pi_train/pi_eval
  -> one common target-size preparation
  -> optional paired-seed diagnostic (recommendation or typed no-recommendation)
  -> operator-owned provisional ordered collection
  -> cross-validate atomic collection freeze
  -> per-frozen-size CV and fresh final production
```

The common preparation is a single authenticated authority, not one independent copy per candidate or fold. A post-selection CV fold may create a fold-local fitted view from its own training partition when its owner requires it, but it cannot create a target-size ladder or alter the frozen collection or that size's exact membership `T_N`.

Foundation-model providers and large accelerator references are released as soon as their final preparation consumer completes. Derived file materialization and target-size candidate views run on CPU/I/O resources unless their current owner explicitly admits an accelerator task. Heavy caches are restored lazily only when a validated artifact is needed.

Candidate execution and continuation

The target-size screen executes only the cells authorized by the reducer's funnel:

```
qualified candidates
  -> coarse n1/M1
  -> at most four short n2/M2 continuations
  -> two final n3/M3 continuations
  -> one recommended size or typed no-recommendation outcome
```

Each `(candidate size, optimizer seed)` cell runs through the accepted TRAIN2 runtime and current EVAL2 owner. A bounded numerical fake may sit below the accepted MACE seam in tests; configuration resolution, target authorities, materialization, checkpoint/provider authentication, persistence, and reducer publication remain production code.

At a fidelity boundary, continuation restores model, optimizer, EMA, LR, and Python/NumPy/Torch CPU/CUDA RNG state. It does not restart from the foundation or substitute an earlier checkpoint. Atomic content-addressed publication means an interrupted boundary either has a complete authenticated endpoint or has no current endpoint. The execution head is reconciled before new work is scheduled, and compare-and-set adoption prevents two workers from becoming current simultaneously.

A boundary interrupted part-way through its matrix keeps its published cells and leaves the reducer at its pre-boundary state, so the whole boundary is still *active* on the next invocation. Active is not the same claim as unexecuted. Before scheduling, the runtime authenticates the durable per-cell progress of the active boundary through the same completion-record replay owner that publication and reconciliation use, reuses the cells that pass, and sends only the genuinely missing cells to TRAIN2 and EVAL2. Recovered and newly executed cells are assembled in exact P2 size-major/seed-minor order, so one complete matrix still produces exactly one reducer transition. Progress that does not authenticate - foreign screen window, a cell outside the active matrix, or a missing or tampered scientific parent - is contradictory durable evidence and fails closed before any new work starts; absence of progress for an active cell is simply work that remains.

Eliminated candidates receive no later ordinary-production authorization. Exhaustive full-fidelity training of every configured size is a separate algorithm/decision-preservation qualification, not a default campaign artifact generator.

Deterministic resource-bounded work queue

CPU-heavy independent tasks use a shared queue with explicit CPU, memory, and I/O ownership. Its responsibilities are to:

- bound executing, ready, in-flight, and buffered work;
- reserve persistent memory before admitting temporaries;
- propagate deterministic task identities and exceptions;
- permit arbitrary completion order where scientific order is irrelevant;
- restore canonical reduction and commit order where FP64 arithmetic or record order is authoritative;
- expose progress and resource telemetry without placing telemetry in scientific identity.

Submission may run ahead to hide hand-off latency, but simultaneous execution remains within the declared resource scope. On NUMA systems, node-local queues, affinity, local stealing, and bounded cross-node stealing are valid execution extensions after measurement; they cannot change canonical membership or reduction order.

Staged evaluation and provider lifetime

Current staged evaluation uses bounded CPU preparation, one admitted accelerator owner when applicable, and bounded CPU finalization. The parent execution owner enumerates the authenticated endpoints, workers perform only their assigned preparation/inference/finalization, and the parent validates run, checkpoint, selected-binding, prediction, and metric identities before durable publication. Fresh and cache-backed endpoints converge through the same parent validation.

Provider scopes are explicit and non-overlapping:

```
candidate provider acquire -> candidate inference/replay consumers
  -> candidate close in exception-safe cleanup
  -> foundation or next provider acquire only after closure
  -> foundation close in exception-safe cleanup
```

Post-selection CV and final-production providers are likewise closed at their owner boundary, including failure paths. A replay cache stores scalar/content evidence, not a live provider. Garbage-collection timing or allocator cleanup does not replace provider retirement.

A worker-private provider shell may be reused only when checkpoint bytes, model class, state keys/shapes/dtype, weight-independent runtime architecture, geometry workload, device, and backend policy all authenticate as compatible. Weight-dependent calculator state is invalidated on replacement. Corruption or authority mismatch is fatal rather than a fallback to an unqualified shell.

One authority per semantic input

The bounded execution representation is:

```
one canonical frame/feature authority
one neutral statistical substrate
one P_train/M3 split and pi_train/pi_eval
one common preparation
prefix views for candidate rungs
training and CV artifacts only for authorized work
```

Memory/storage must not scale as one product-sized descriptor, graph, or membership copy per target-size rung. Descriptor shards, fixed-file views, replay indexes, and frame caches are reconstructible only when their content

and recipe identities authenticate. A cache hit is never a substitute for the selected binding or another scientific authority.

Memory, storage, and scratch admission

Long stages account for

$$M_{\text{stage}} = M_{\text{persistent}} + M_{\text{inflight}} + M_{\text{buffered}} + M_{\text{sparse}} + M_{\text{result}} + M_{\text{scratch}}.$$

New work is admitted only when CPU, RAM, accelerator, disk, and scratch reservations fit the stage plan. The live ledger is authoritative: a prospective target-size or evaluation reservation replaces only the exact modeled reservation it supersedes and preserves all other live owners. When retained growth is not bounded, sequencing is conservative rather than relying on an optimistic projection.

Large reconstructible arrays may use mmap/file-backed persistence. Atomic publish-or-validate-winner rules protect concurrent fixed-file and materialized cache creation. Stale, corrupt, or mismatched caches are rebuilt; they are not silently accepted as evidence.

Persistent campaign state uses a compact SQLite store, append-only event history where needed, content-addressed files for large payloads, and completion records written only after required artifacts are durable. A restart distinguishes complete, incomplete, stale, corrupt, and superseded state and re-authenticates currentness before reuse. Cleanup removes only known campaign-owned reconstructible state and preserves external inputs, selected scientific records, restart checkpoints, and diagnostics needed for recovery.

Storage and I/O management

Storage is a first-class resource plane and never a second scientific authority. `mdstats.training_data.storage` turns each accepted current owner into a uniform *owner view* and composes those views into one cross-owner inventory. Semantics come from the owning API; pathnames, report labels, stage names, process ids, and file ages carry no authority at all.

Authority is invocation-local. `--apply` on the invocation being run is the only thing that authorizes a mutation, and the subcommand being run is the only thing that selects the action; an `apply` or `action` key under `[storage]` is rejected rather than obeyed, and no environment variable is consulted. The complement is that every non-apply path is genuinely observational: it creates no workspace, no state database, no generation root, no control plane, no acceleration receipt, and no report artifact.

Observation is an invocation-scoped capability carried by a context variable, not a flag on the first store a command opens. It reaches nested owner helpers and the worker threads the storage fan-out spawns, so no helper can escape it by calling an ordinary default-creating constructor; and it is enforced as well as declared, because an observational campaign-state open is a read-only SQLite connection whose write paths refuse before committing. Nothing process-global is toggled to achieve it, so a concurrent consequential operation keeps its own writable store and receipt behavior.

Every consequential mutation follows one path:

```
real P1-P7 owners -> owner views -> cross-owner inventory snapshot
 -> resolved storage policy -> immutable owner-bound plan
 -> owner publication barrier + revalidation -> executor -> durable audit
```

Retention is a transitive closure, not a per-owner question. The current P7 publication is a read-only descendant of the accepted P5 publication and re-authenticates the exact P5 checkpoint bytes at their canonical hot paths, so those bytes stay pinned after the P7 attempt retention reference is released. P4′s current terminal authority pins the P3 evidence its canonical loader needs. A truthful `waiting_for_reference` pins the whole

predecessor lineage. Protection is monotone: no owner's cache or history classification overrides another current owner's requirement, and the closure is rebuilt from live owner records rather than persisted as a second registry.

Mutation is race-safe, not merely recent. P5 and P7 both publish an immutable object and then the pointer that makes it current, so there is a real window in which the object exists and nothing references it. Each owner exposes a per-generation publication barrier that the publisher holds across both steps and that any storage mutation acquires across revalidation and mutation. The storage-operation lease serializes storage against storage only, and is never mistaken for serialization against the owners.

Completion is proved by a retained anchor. When a post-selection run reaches its terminal record, P5 freezes its completion proof as two create-once records: an immutable topology manifest naming every node the run produced, published first, and a compact self-authenticating anchor binding that manifest's identity, published last as the commit point. The split is what lets normal reporting validate completion in O(1) while exact closed-subtree certification still pays for the full topology. The topology is typed and covers directories as well as files, so neither an unexpected empty directory nor a same-name file/directory substitution can pass as the node the owner certified, and no symlink or special object becomes owned by appearing at a familiar name. Every observation on that path is no-follow, and the owner's own authority records are opened with `O_NOFOLLOW` and confirmed regular by `fstat` on the opened descriptor rather than by a separate `lstat` a rename could invalidate. From then on the anchor - not the presence of the terminal evidence file - is what certifies the run. The distinction matters because the terminal evidence is an ordinary archive member: an interrupted cold reclamation may already have moved it, and a certification that needed it would leave that reclamation unable to finish. Both records are owner infrastructure, never part of the reclaimable member set. Republication verifies and reuses the existing proof rather than deriving a new one from a tree storage has legitimately depleted, and a tampered, copied, or self-inconsistent proof makes the run non-certifiable instead of appearing to own more.

A released P7 attempt proves its own scratch. Releasing an attempt publishes a versioned typed topology proof bound to the exact released state, written before that state so the state stays the commit point; an aborted attempt that legally reopens as active invalidates its release proof for free, because the state it bound is no longer current. Every top-level node must be one the proof recorded, of the recorded kind, before storage exposes it as reclaimable. An attempt state that cannot be authenticated is not skipped: its references can pin exact P5 checkpoints, so it becomes an owner-graph integrity failure that blocks consequential planning until repaired, and the retention fence independently denies destructive authorization for every campaign-managed path while the ambiguity lasts - the lost references routinely name artifacts outside the P7 tree, so protecting only that tree would leave the unknown asset authorizable.

Authentication is strict, root-bound, and performed by one authority every storage-facing consumer reads. An attempt counts as authenticated only when the enumeration reached it without traversing a substituted name-space component - no-follow at the generation root, the `attempts` container, and the attempt root, not merely at the state file - and when its persisted digest recomputes, its recorded identity matches the directory, and that identity is the canonical identity derived from the qualification binding the state names. Enumeration is by actual attempt directory, so an attempt with no state at all is visible rather than absent.

Containment is not ownership. A directory owner view declares one of two coverage semantics. A *closed subtree* is one whose real owner certifies, from its own authenticated record or exclusive-writer contract, that every traversable descendant belongs to that artifact; a *container* is owner-known but its descendants need individual views, and anything unknown beneath it stays ambiguous and retained. Only a freshly revalidated closed subtree may be recursed into destructively. P5 records a run-member manifest when a run reaches its terminal record, because the run directory is delegated to the configured trainer; P7 records an attempt-member manifest at the moment an attempt becomes terminal; the campaign store's externalized record area is closed by exclusive-

writer contract. A superseded target-size execution root records no such membership and is therefore honestly a container. A nested mount below an authorized root is a further ownership boundary and is never traversed.

Archive is representation, not resolution. Hot bytes are replaceable only for owner-declared historical bulk with no current or restartable hot dependency; no P1-P7 loader is given an implicit cold-read fallback. Archive A reclaim or restore additionally binds the exact retained representation it intends to consume and re-authenticates that catalog entry, manifest, and blob *inside* the protected consequential window, before removing a hot member or installing a restored one; every supported writer of retained archive control state takes the same storage-operation lease, which is what makes that check race-closed. A restore also binds the `(device, inode, type)` of every existing parent it installs through, so a same-path directory swap refuses rather than redirecting the installation. Archive verification and restore bound member paths, member types, member count, total expansion, per-member size while streaming, and decompression amplification before writing anything, and a manifest carries an identity-owned relative locator resolved only inside the storage-owned archive root. A requested root may narrow a selection into an eligible artifact but never widen it to an ancestor, an archive identity binds its representation (codec, level, serialization) and not only its logical content, and a restore is an exact owner-bound plan that never metadata-mutates a container that already existed. Terminal catalog and restore receipts are published only downstream of flush, atomic publish, directory-entry persistence, and authentication of the published bytes.

Audit publication and retention are one serialized lifecycle. Both happen under the storage-operation lease, so bounded retention cannot rewrite away a record another operation just published and reported as durable. The stored record states its own successful publication; retention refuses to rewrite over a damaged stream; and a retention failure is surfaced separately without unpublishing anything or touching the mutation.

Deduplication is direct inode sharing under an owner contract. Byte-identical members share one inode among themselves. The pre-rename alias is staged in storage's own operation-scoped staging area rather than inside the owner's run, so a hard crash leaves storage-owned residue with a recovery lifecycle instead of an unrecorded descendant that would block future certification; abandonment is established by the storage-operation lease and the journals, never by a process id or an age. There is deliberately no persistent content-addressed store, which would be a second durable copy of campaign bytes with its own retention lifecycle. Exact byte equality is necessary but never sufficient: file type and owner-required metadata must match, the canonical member's link count must be fully accounted for inside the group, the family must have no accepted in-place writer, and cross-device or unsupported filesystems retain duplicate bytes without a correctness failure.

Reporting is bounded and complete. The normal report costs one `lstat` per declared owner artifact and never walks a subtree, so directory aggregates are labelled unknown rather than guessed and `--deep` is the explicit opt-in to exact recursive physical accounting. The census is complete: an unrecognized workspace tree is reported as ambiguous and retained rather than omitted or pooled.

Campaign-state maintenance is two planned actions. Bounding diagnostic events and rewriting the state database are separate authorities. Excess events authorize pruning only, executed at exactly the resolved bound with no hidden floor - a small transaction that takes the write lock up front and so serializes against any other campaign writer. A rewrite is planned only when a fresh measurement already satisfies the configured reclaimable threshold, and re-establishes that threshold and its temporary-space admission inside a cross-process exclusion that every campaign-state writer participates in - writable construction included, since schema bootstrap is a real write - and that it holds through the rewrite. The gate is one per database shared by every store instance in the process, its reentrancy belongs to the acquiring thread rather than to an object, and its advisory lock file is campaign-store infrastructure no storage action targets; a second process consuming the free pages while maintenance waits therefore refuses the rewrite instead of performing an unjustified one. Free pages that pruning

created do not widen the prune into a rewrite; that belongs to the next fresh plan. A refused or empty cleanup can never carry either along, and results distinguish `events_pruned` from `vacuum_performed`.

Storage owns durable state of its own - an identity-keyed archive catalog, manifests and blobs, restore journals, a bounded execution audit, and operation-serialization state - under an explicit control-plane root. Terminal restore journals are retained to a bound while a nonterminal one is recovery authority, and catalog fields that establish what a representation *is* are create-once. None of it carries a currentness decision, and none of it can be reclaimed while a retained cold representation still needs it.

GPU/VRAM and host admission

GPU jobs are admitted against explicit device availability, free memory, and configured budget evidence. A one-job calibration establishes whether the applicable serial workload is viable; it does not by itself authorize parallel expansion. Soft utilization and fractional-VRAM envelopes regulate additional jobs both upward and downward, while independent hard conditions protect against OOM. Missing telemetry at calibration startup selects conservative serial execution when the device is otherwise usable; it does not create parallel evidence.

Training admission additionally recognizes infeasibility. CUDA training starts with one job only when one job is currently resource-admissible; zero safe admission is a valid execution state rather than a floor to be rounded up. Current aggregate occupancy counts regardless of which process owns it, a configured minimum concurrency is subordinate to current feasibility, and a positive configured job count is a maximum cap rather than launch permission. Pending training work with an idle queue and no feasible slot resolves to an explicit resource failure, not a launch or a wait. Device availability and memory observability are separate facts, and absent a trustworthy current memory observation automatic training admission is blocked. Memory safety is evaluated on every trustworthy sample independently of optimizer/epoch calibration readiness, which remains the prerequisite only for estimating scalable demand. A training slot owns training lifetime alone; post-training evaluation must not inherit a training slot. The evaluation/inference controller keeps its own accepted serial-floor calibration contract, which these training rules do not replace.

For training, the configured `training_gpu_memory_fraction` is the admission ceiling *and* the live aggregate **soft** admission/backoff boundary. It is a control boundary, not a scientific or execution verdict: crossing it is never by itself terminal memory infeasibility. One trustworthy observation at or above that envelope immediately blocks any further admission or promotion at every active-job count. That single observation is only a candidate: it may be rechecked once so an allocator fluctuation does not stop a run.

Training admission is therefore symmetric. The controller admits upward one job at a time on sustained true-epoch evidence, and it backs off downward one job at a time on sustained aggregate pressure. If the next normal control observation is still at or above the envelope while more than one owned training job is active, that persistence proves *the current concurrency* is unsafe, not that the workload is infeasible. The scheduler retracts exactly one prior admission: the most recently admitted currently active owned job is demoted (reverse-most-recent-promotion), which reuses the existing admission ordering rather than any victim-selection subsystem. Unaffected active jobs keep running. Backoff proceeds one level at a time - `3 -> 2`, then `2 -> 1` only on fresh persistent evidence at two - so every transition has its own observable causal evidence.

A demoted job's cooperative stop is per job. The scheduler hands every admitted job its own stop handle; backoff sets exactly one of them, and a terminal abort sets every one of them, which is what whole-wave cancellation means. A future cancellation request is not teardown: the scheduler blocks until the demoted worker has actually returned, so the child process has exited and the run's own finalization has run, and only then does it re-observe device occupancy and make the next scheduling decision. No replacement admission, requeue, or restart may be ordered before that boundary.

How long that teardown may legitimately take is owned by the process owner, and the scheduler imposes no deadline of its own on the demoted future. That future is the whole run: when the stop is requested it may still be in run-owned preparation, recovery classification, or materialization and may never have reached the trainer, so no subprocess-termination clock describes it and elapsed time there is not evidence about owned teardown. Child termination is instead bounded where the child is owned: the process owner escalates SIGINT, one termination grace, SIGTERM, one termination grace, then an unconditional SIGKILL and reap, so a stopped child always terminates inside its owner and whatever verdict that produces reaches the scheduler as the future's own outcome. Optimizer-activity freshness (`parallel_training_epoch_activity_timeout_seconds`) remains purely a child progress-liveness bound and has no authority over process teardown, so changing it cannot change resource-safety semantics, and no operator-facing teardown knob exists at all.

The same per-job stop handle is also read at run-phase boundaries that precede the trainer, so a slot demoted while still preparing or materializing stops spending effort it will not use instead of launching MACE. That is the one cancellation mechanism, not a second one: those boundaries sit before any partial fold evidence exists, they leave the run root under the existing materialization/checkpoint authority, and they produce the same explicit cancellation outcome the trainer produces, so the scheduler classifies them as an ordinary retractable demotion.

A resource demotion is not a scientific run failure, but only the execution owner may say that a demotion is what happened. The trainer reports an explicit cancellation outcome when - and only when - it observed the requested stop and terminated its child through the normal termination/finalization path; a supervisor's intent to stop a job is never evidence about why that job raised. On that explicit outcome the demoted task returns to the pending/restartable queue with its frozen slot identity, is not counted as a failed job, publishes no partial fold, and resumes later through the existing checkpoint/continuation authority rather than any retry identity or second checkpoint convention. Completed folds stay completed, and the planned folds still complete exactly once. Any other exception from a job selected for demotion - a backend fault, a nonzero MACE exit, a CUDA allocation failure, an interrupt, or a programmer error that races the stop request - keeps its own authority, is counted as a failed job, and ends the invocation through the terminal path instead of being requeued.

A concurrency level that live telemetry has disproven lowers a scheduler-owned effective ceiling to one level below it for the rest of the current post-selection execution, and that ceiling is monotone downward: the same execution cannot oscillate back into a level it already falsified. The ceiling is runtime control state only - not campaign configuration, not durable scientific evidence, and not a persisted hardware profile. The per-job VRAM estimate is advisory admission input; live aggregate telemetry is authoritative for runtime adaptation, so an estimate the device later disproves is a reason to adapt concurrency rather than to fail.

Terminal memory infeasibility sits outside this adaptation loop. It may be declared only when no lower owned concurrency state remains capable of resolving the problem - that is, after convergence to the minimum executable concurrency - or when an independent authoritative hard condition applies: a backend/device allocation failure, a failure of owned teardown/reclamation to re-establish a safe owned execution state, or another established hard-failure invariant such as sustained loss of live memory observability. Raising the configured envelope is not a repair for concurrency pressure. GPU-utilization saturation stays soft in the same way: while memory itself remains inside the envelope, saturation lowers the replacement target and never stops running work.

Live memory observability is a precondition for continuing to own accelerator work, not merely for promoting it. While training is active, a missing current memory observation admits and promotes nothing. One isolated missing observation is tolerated only when the immediately preceding trustworthy observation was safe; a second consecutive control observation without a trustworthy sample, or a lost observation immediately after an unsafe one - where recovery can no longer be established - is a terminal resource-observability failure for the current training wave. A later invocation retries normally once observability is restored. This is expressed in the existing

control loop; no second telemetry thread, monitor daemon, or persisted observability state is introduced, and the bounded transient tolerance is controller-local rather than operator configuration.

Any exception escaping the training scheduling wave ends the invocation. It stops new admission, signals every owned child's stop handle, reaps every owned active child through the existing supervision path, and is re-raised before any post-training evaluation begins - including for previously completed sibling slots. A device whose training state is unknown or already unsafe must not receive fresh accelerator work in the same invocation. Progress is not lost: authenticated training summaries and their materializations remain durable restartable state, and the next healthy invocation resumes outstanding training/evaluation work through the ordinary continuation path. No out-of-memory stderr classifier, retry database, or alternate handoff record is involved.

A transient architecture or classification realization is not training. A temporary accelerator model built to answer a recovery or currentness question is retired at its own ownership boundary, including on failure paths, so it cannot contribute residency to the baseline a later admission decision is measured against.

An execution controller may lower concurrency after measured resource pressure, but it cannot change scientific batch/exposure semantics, precision policy, checkpoint evidence, or target/replay membership to fit memory. OOM recovery is valid only when the retry is protocol-equivalent and the changed parameter is non-semantic.

Replay indexing and bounded parsing

The selected replay source remains external scientific authority. A reconstructible index may store source-byte identity, frame offsets/lengths, atom counts, and source-order geometry identity for sparse monitor access and bounded chunk parsing. Source mutation or index corruption causes safe reconstruction. Parser concurrency is added only when representative measurement shows benefit and exact replay bytes/identities remain unchanged.

Progress and observability

Every long-running stage exposes scientific progress and executor state:

1. completed/total work and percent where meaningful;
2. elapsed time and ETA when estimable;
3. throughput with an explicit stable unit;
4. active, pending, or buffered work;
5. resource pressure or the current hot item where relevant.

Heartbeats are emitted during long periods without task completion. ETA is based on globally committed work. User-facing elapsed and known ETA use fixed `HH:MM:SS`; unavailable ETA is `--:--:--`. Presentation state never enters scientific digests.

Performance qualification boundary

Performance changes are compared on representative work with equivalent scientific inputs and runtime conditions. Evidence records wall/CPU time, throughput, RSS/VRAM, scratch/storage, queue/backpressure, and output digests when material. A speedup obtained by changing precision, evidence population, ordering, or output is not a conforming optimization.

Target-machine GPU and long real-production qualification remain separate from P6 functional closure. They require their own supported hardware, workload, backend, and acceptance evidence; the current campaign does not infer those results from CPU or bounded numerical tests.

Part VII - Ownership and extension boundaries

One current-generation authority model

The current campaign has one semantic generation. A record, policy, or artifact is either authenticated for that generation or unsupported. Historical selector, repair, migration, and campaign-generation formats are not alternate current execution paths.

Architecture owns durable structure and scientific/algorithmic invariants. Specifications own exact schemas, policy values, tolerances, failure codes, and module-local behavior. Workplans coordinate proposed transitions and never become product authority merely because implementation follows them.

The core authority chain is:

```
source evidence and labels
    -> eligibility / conditions / evidence roles
    -> neutral statistical substrate and protected relations
    -> one P_train / M3 split
    -> one pi_train and nested pi_eval ladder M1 subset M2 subset M3
    -> one common target-size preparation
    -> optional paired optimizer-seed automatic diagnostic (recommends only)
    -> operator-owned provisional design (ordered collection of (N, CV horizon, production
horizon))
    -> cross-validate admission
    -> frozen design: every selected size N_selected, its exact T_N = pi_train[:N_selected],
and role horizons
    -> post-selection cross-validation on the frozen collection
    -> fresh final production on the complete selected dataset(s)
    -> currentness-fenced publication
```

There is no branch to a second membership selector, a per-domain target-size map, a generated-size rescue, or downstream-evidence-driven fallback. Derived state from an unsupported generation is rejected before semantic reuse and is quarantined/reprepared rather than translated.

Scientific decision ownership

Decision or product	Sole current owner	Consumes	Emits	Explicitly does not own
source/label identity	DATA2-family contracts	immutable source material	normalized labelled-record identity	partition, selection, training
conditions/eligibility	DATA3-family contracts	source records	eligible frames and conditions	evidence-role assignment
raw features/events	DATA4-family contracts	eligible evidence and provider declarations	partition-independent raw evidence	fitted metrics or membership
evidence roles and protected relations	DATA5 partition contracts	cohorts, independence evidence, purge rules	neutral roles (development/monitor/calibration/locked) and split exclusions	post-selection CV folds or target ranking
descriptor/difficulty inputs	DATA6 contracts	authorized evidence and frozen foundation model	raw/blinded descriptors and predictions	target membership
common fitted preparation	current P3 preparation owner	neutral substrate and frozen method inputs	transforms, metrics, E0, objective/weights, difficulty inputs	membership or target size
target-size split and orders	current target-size experiment owner	frame authority, neutral substrate, configured policy	`P_train`/`M3`, `pi_train`, `pi_eval`, `M1`/`M2`/`M3`	method acceptance
common target-size preparation	`TargetSizeCommonPreparation`	`P_train` and foundation/training protocol	one shared preparation identity	per-size or per-seed scientific variation
automatic target-size diagnostic	one target-size reducer	paired target-side screen evidence	a *recommended* size, or a typed no-recommendation outcome	freezing a size, monitor cardinality, CV evidence
provisional downstream design	operator, through `select-target-size`	qualified candidate set, `pi_train`, configured/overridden horizons	one mutable ordered collection of per-size entries (`N`, `T_N` identity, `H_cv`, `H_prod`), unique by `N`	immutable ancestry; running screen work; choosing a release product among sizes
frozen downstream design	`cross-validate` admission	the current proposal and authenticated P2 order	frozen ordered collection of per-size bindings (`N_selected`, exact `T_N`, role horizons)	re-deciding size afterwards
post-selection method acceptance	post-selection CV owner	frozen target collection, protected relations, `K >= 2`, CV dataset, required seeds	all-required-fold target-only verdict across each admitted size	changing selected sizes

A narrow specification may refine a row's realization, but it cannot create a second semantic owner for the same decision.

Fitted preparation boundary

The current fitted-preparation owner may publish:

- heterogeneous feature transforms and metrics;
- foundation predictions and training-domain difficulty evidence;
- atomic-reference/E0 fits;
- training objective and configuration/property weight records;
- condition, provenance, event, environment, and diversity inputs;
- immutable identities linking products to their authorized inputs.

These are inputs to the one canonical order. They are not an independent quota, FPS, membership, target-size, or CV selector. A fold-local transform is allowed only after selection and only when the post-selection CV owner binds it to that fold's training partition; it cannot change the frozen target collection or that size's exact membership `T_N`.

This boundary preserves useful fitted/statistical information without reintroducing a domain-specific target-size authority. Materialization and export records describe consumer views and remain downstream of the selected binding.

Target-size authority

Let (N_{}=|P_{}|). The candidate ladder is a configured contiguous power range,

$$\mathcal{N}_0 = \{2^p : p_{\min} \le p \le p_{\max}\},$$

with materializable population

$$\mathcal{N}_M = \{N \in \mathcal{N}_0 : N \le N_{\text{available}}\}.$$

The qualified population is the subset admitted by the current target-size policy. The selected size must be a qualified member. There is no hidden scientific ceiling beyond the configured policy and available population.

Every candidate is an exact prefix:

$$T_N = \pi_{\text{train}}[:N].$$

Thus candidate sets are nested prefixes of the canonical training order, and for every admitted size `N_selected`, its exact prefix `T_N = pi_train[:N_selected]` is frozen together with its role horizons. Increasing `N` only adds frames; a pass/fail/pass result under a monotone prefix policy is an invariant failure, not a reason to choose a different order.

The reducer consumes only authorized target-side development/model-selection evidence. Replay metrics, post-selection CV, calibration, physical-observable, and locked-test evidence cannot rank, reject, or tie-break a target size. Fewer than three qualified sizes is a typed no-recommendation outcome.

What the reducer emits is a **recommendation**. It is short-horizon evidence about target-force RMSE under one configured screening protocol - not proof of asymptotic target-size convergence, long-horizon training quality, final model quality, or MD stability. The operator owns the actual decision, and may accept, ignore, or override the recommendation at any time before `cross-validate` freezes the design.

The configured ladder ceiling is a **practical budget limit**, not a requirement that convergence occur below it. At the terminal comparison the current terminal-decision policy (`practical_equivalence_then_practical_ceiling.v2`, bound into P2 policy identity) is:

- if two finalists differ by no more than `practical_equivalence_mev_per_a`, the smaller finalist is recommended – a raw improvement at `Nmax` inside that band is a plateau, not unresolved convergence;
- if a smaller finalist has the best terminal paired-mean target-force RMSE, it is recommended normally;
- if `Nmax` is materially superior to every other successful terminal finalist by more than the practical-equivalence threshold, `Nmax` is **recommended** and the result carries the non-blocking warning code `nonconverged_at_configured_ceiling`, meaning that the configured practical ceiling is the best evaluated permitted size while a plateau was not demonstrated within the configured ladder. It does not claim that `Nmax` is asymptotically converged.

A ceiling recommendation follows the ordinary diagnostic path; no separate status, lifecycle, or admission path exists for it. Genuinely insufficient comparison – too few complete comparable finalists, or malformed, missing, duplicated, reordered, or lineage-incompatible boundary evidence – remains a no-recommendation outcome and is never converted into a ceiling recommendation. No unconfigured rescue size is invented. A no-recommendation outcome is a conclusion about the diagnostic, not about the campaign: it leaves any existing proposal untouched and does not prevent an explicit qualified choice. Evidence reduced under the retired blocking-ceiling rule stays historical and is never relabelled.

Post-selection ownership

The current post-selection graph is:

```
current selected binding
    -> shared method identity
    -> CV policy / final-production policy
    -> CV plan / final-production plan
    -> fold/final evidence
    -> final-production publication decision
```

Cross-validation evaluates the frozen selected collection, preserves P1 protected relations, requires every configured fold and seed, and accepts or rejects the method. It cannot alter the frozen target design. Final production starts fresh from the accepted foundation and trains the complete selected dataset(s) under each size's own frozen production horizon `H_prod_i`; it cannot continue a screen or CV run.

The final-production publication decision

Deciding *which* completed production seeds constitute the released product is the last pre-qualification act, and it is owned here rather than downstream. `train-production` takes it immediately after the required seeds complete, when every input it needs already exists and no downstream release evidence does. If the decision were taken later, "the committee" would silently become "the members that survived qualification" - member selection on release evidence.

Each completed run durably publishes the exact records that chose its representative: the representative EVAL2/admissibility record and its M3 target metric record. Those were previously referenced by digest only, which left no authenticatable basis for any cross-seed decision. A run root written before they were durable is *re-evaluated* through the real EVAL2/provider owner on its exact authenticated checkpoints and must reproduce the digests its run evidence already bound; nothing is ever synthesized from a digest.

The decision record binds the selected binding, the final plan and policy, the accepted CV/method lineage, the frozen M3 membership, every required seed's run evidence and representative identity, the canonical target head, the committee policy, the exact ordered published member set, and a deterministic decision-policy identity. Both configured policies are supported:

- `all_qualified_final_seeds` publishes every required seed whose already-frozen representative is admissible under the accepted checkpoint policy;

- `single_best_final_seed` ranks only those already-frozen representatives with the accepted target-only EVAL2 ordering over the common frozen M3 evidence, with tie material descending from the final-production plan identity, and publishes the first canonical admissible representative.

No downstream metric, target-size statistic, physical score, or locked score participates, replay evidence remains admissibility-only, and there is no API that adds, removes, or reorders a member afterwards. A decision that no longer binds the current lineage stays on disk as historical evidence and is unreachable as the current product.

Every current consumer re-resolves the selected binding and current campaign revision before exposing a descendant. A stale caller-held object, checkpoint, or provider cannot become current. Publication rechecks currentness in the same transaction that would install a current pointer.

Public orchestration and storage boundary

The public scientific lifecycle is exactly:

```
init -> doctor -> prepare -> select-target-size -> cross-validate -> train-production
```

`prepare` builds the neutral/current substrate and common preparation but selects nothing. `select-target-size` owns the provisional downstream design, which is an ordered collection of distinct qualified sizes over that one prepared generation: `<N>` merges a qualified candidate into it - appending a new size, or replacing an existing size's complete entry in place - and trains nothing; `--auto` runs or reuses the automatic diagnostic and merges its recommendation through the same owner; `--reset` clears the design pre-freeze; and `--horizon-cv` / `--horizon` steer the two role horizons of the size the invocation touches. It freezes nothing. `cross-validate` owns the atomic whole-collection freeze and then selected-only method acceptance for every frozen size. `train-production` owns the collection-wide cross-validation barrier and then one fresh final publication per frozen size. `status` and `advance` project these same owners; they do not create another state machine. `storage` is orthogonal: it manages representation, retention, caching, archival, and admission, and it advances no scientific lifecycle.

Post-production qualification is a separate downstream family and is deliberately not part of that lifecycle:

```
qualification status | qualification run | qualification activate-locked
```

`advance` may route ordinary nonlocked `qualification run` once a single-size campaign has a current final publication, because that step is repeatable and consumes only evidence the campaign already owns. It never opens locked evidence: `qualification activate-locked` is irreversible one-shot disclosure and stays an explicit operator act. For a multi-size frozen design the completed experiment is terminal and non-release-qualified, so no qualification attempt is routed or created at all.

Downstream qualification ownership

Deployment parity, physical PES/relaxation/dynamics validation, uncertainty calibration, and locked testing are owned by `mdstats.training_data.qualification`. That package is a *consumer* of the accepted final-production publication, never a second product authority. Its owner graph is:

```
accepted current selected binding (P4)
    -> accepted post-selection CV (P5)
    -> accepted fresh final production and its currentness-fenced completion (P5)
         |  immutable and read-only to qualification
         v
    QualificationInputBinding
      exact publication + ordered members
      + executable candidate identity
      + target-machine environment fingerprint
      + frozen qualification specification
      + frozen neutral evidence-role membership
```

```
     |
     v
ProductionQualificationPlan  (+ candidate-independent PhysicalValidationPlan)
     |
     +-> deployment parity through the supported ML-IAP/LAMMPS runtime
     +-> local PES response against matched external references
     +-> fixed-cell relaxation topology and geometry fidelity
     +-> finite-temperature dynamics stability
     +-> uncertainty calibration, or an explicit not_applicable
     +-> explicit one-shot locked interpolation test
     |
     v
ProductionQualificationRecord -> ReleaseEvidenceIndex
```

The publication resolver is the accepted P5 publication-decision owner; qualification copies that decision's own ordered member set and adds no publication, membership registry, or member-selection rule of its own. Both committee policies are decided upstream, so qualification contains no cross-seed ranking at all.

The exact canonical P5 target head travels with every published member and is part of both the member identity and the deployment identity, so an artifact exported from the replay or foundation head is a different product rather than the same product serialized differently. The deployment export and the ML-IAP builder are both called with that head; neither accepts `None` for a multihead-capable product. Deployed artifacts are published create-once under an advisory per-artifact lock and are re-authenticated from a durable receipt and their bytes before every reuse, including after a process restart - a full PyTorch model pickle is not byte-deterministic, so identity is carried by the receipt rather than inferred from the bytes.

Every public qualification resolver re-establishes the current `QualificationInputBinding` at exposure time and validates the located object against it. The campaign-store pointer is a locator only: a terminal verdict published under an older specification, executable, environment, or product is historical, never current, and `qualification status` cannot print it as a current release verdict. Locked disclosure history is deliberately kept outside that fence, in an append-only reveal index, so a currentness change can make a verdict historical but can never make a revealed cohort fresh again.

Reference-dependent components are keyed by a component-input identity that includes the exact frozen request and the exact authenticated bundle, so replacing a bundle under the same request stales local PES, relaxation, and dynamics while leaving deployment and calibration evidence reusable. Dynamics runs from the authenticated reference-relaxed coordinates of each physical base, never from the unrelaxed base geometry, and its reducer - not the runtime worker - decides NVT/NVE temperature behaviour, energy drift, safety bounds, and protected topology, displacement, bond, and angle degradation under thresholds frozen before execution, including an explicit consecutive-sample persistence rule that separates transient noise from real damage.

Locked activation is an irreversible *open* event rather than proof the evaluation completed. A crash between opening the cohort and publishing the result is resumable onto the same activation identity; only a genuinely terminal result makes a second activation a rejected duplicate.

Qualification concurrency and nested thread budgets come from the accepted campaign resource owner, and the resolved resource scope is bound to the attempt separately from the numerical environment identity, so machine capacity is recorded without making a deterministic numerical claim machine-specific while a materially different resource scope still cannot silently reuse a performance claim.

That scope digest is identity, not measurement. Each attempt also publishes one immutable resource observation - total and per-component elapsed time, workspace filesystem total/free bytes and the attempt's own footprint at start and end, the configured `[execution].minimum_free_disk_gib` reserve and whether it held, peak process

RSS, and accelerator model/VRAM where an existing owner reports them - which the terminal record and release index both point at. Those observations are evidence and never stale numerical results; their one operational role is that an attempt which cannot satisfy the existing disk reserve aborts before materializing work rather than changing any scientific input. Reading that reserve is an owner-local safety check, not the storage admission plane.

Stress applicability is likewise a capability decision rather than a configuration switch: it is resolved before execution from the accepted training objective's stress weight, reference stress labels, whether the authenticated model returns a stress tensor, periodicity, and runtime support. Policy may require stress or record a justified inapplicability reason, but it cannot relabel an available trained channel as `not_applicable`. Each source converts to canonical ASE/MACE Cauchy stress in eV/Angstrom^3, positive in tension, exactly once; units and sign belong to the source adapter, so LAMMPS `units metal` thermo pressure - bar, positive in compression - is converted only by its own named adapter and is never parameterized by a caller.

The exact three-axis periodicity vector is carried through every deployed request, the LAMMPS boundary command, the raw observations, and the dynamics case identity. A mixed boundary is executed as itself or fails closed; it is never coerced to fully periodic or fully open, and minimum-image reductions wrap only the axes that genuinely have images.

Downstream evidence has pass, reject, and waiting authority for the exact frozen product and nothing else. A failure never changes the frozen target design, CV acceptance, a production checkpoint or seed, publication membership, or an upstream threshold. A missing external reference is `waiting_for_reference` with an actionable request on disk, never a fabricated pass. An absent supported deployment runtime is reported as unavailable and blocking, never as either a pass or a scientific rejection.

The locked interpolation test is opened only by `qualification activate-locked --confirm`, only after every mandatory nonlocked component has passed, and only once for a given publication and locked cohort. After activation the revealed cohort is never a fresh locked test again, whatever the policy is changed to.

Physical numerical algorithms remain owned by their analysis modules. A downstream recipe must bind matched collection/frame identity, runtime and capability identity, analysis-owned results, and a declared statistical role.

Qualification persistence and the successor-storage handoff

Qualification evidence lives under one canonical generation-scoped root, `<workspace>/.mdstats/qualification/g<N>/`, with `objects/` holding immutable create-once release evidence and `attempts/<attempt-identity>/` holding attempt-local state and bulk scratch. Currentness is never persisted as a second truth: it is re-established through the P4/P5/P7 owners and published as a generation-fenced pointer in the campaign store, exactly as P5 descendants are.

The storage subsystem consumes these owner entry points and needs no pathname inference:

```
CampaignStore                          current campaign state owner
P3 target-size generation/root owner   execution evidence and reconciliation
P4 selected binding owner              current selection authority
P5 post-selection root/store           CV, final plan, run completion
P7 publication resolver                resolve_authenticated_final_publication
P7 qualification root/store            qualification_root, QualificationEvidenceStore
P7 terminal result owner               ProductionQualificationRecord, ReleaseEvidenceIndex
P7 attempt/retention owner             QualificationAttemptState, QualificationRetentionFence
P1 frame-cache owner                   the one exact-reconstruction cache seam
CampaignStore receipt cache            stat-keyed SHA-256 acceleration only
```

Qualification adds no cache authority, no second cleanup policy engine, no global retention registry, and no part of the storage inventory, archive, deduplication, or admission plane. Its retention reference is coordination metadata

only: it says that one already authoritative artifact is actively referenced by an in-flight attempt, it is released on terminal completion or explicit abort, and it can never make a stale publication current.

Unsupported generations and compatibility

Current loaders do not semantically read or migrate obsolete target-size derived state. The narrow cutover detector may inspect record names or minimal generation metadata solely to reject before candidate/checkpoint reuse. It may quarantine the opaque record under a namespace no current loader reads, but it cannot decode, reconstruct, normalize, or bind that payload into current authority.

Independent lower-level source/frame/content caches may be reused only after their current owners revalidate source bytes, recipe, lineage, and integrity. Compatibility readers for non-target product responsibilities remain read-only and non-authoritative. Historical schemas and rationale belong under `docs/history/mlff/`; their presence does not create a current API.

Extension boundaries and summary

A future extension may enrich neutral feature/preparation inputs, extend the canonical `pi_train` policy, add an explicitly qualified screen metric, or change execution representation only when one-owner direction, exactness, protected evidence roles, and currentness remain intact. A new campaign generation is not entitled to automatic migration support.

The durable rules are:

1. independent evidence remains independent;
2. fitted preparation and target membership are separate authorities;
3. one canonical order and one common preparation define every candidate;
4. the ordered collection of `(N_selected, T_N, H_cv, H_prod)` bindings is frozen once at `cross-validate` admission;
5. the automatic screen recommends and the operator decides; post-selection cross-validation accepts the method and can never re-choose the size;
6. final production is fresh full-selected-set training;
7. execution/cache/provider choices cannot change scientific semantics;
8. unsupported derived state is rejected before reuse rather than migrated;
9. downstream qualification cannot become a selection fallback: it validates one already frozen product and has no path back into selection, CV, production, or publication membership.

References

[1] I. Batatia, D. P. Kovacs, G. N. C. Simm, C. Ortner, and G. Csanyi, "MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields," *Advances in Neural Information Processing Systems* **35**, 11423-11436 (2022). DOI: 10.48550/arXiv.2206.07697.

[2] ACEsuit, "MACE descriptors," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/descriptors.html (accessed 2026-07-27).

[3] H. Flyvbjerg and H. G. Petersen, "Error Estimates on Averages of Correlated Data," *Journal of Chemical Physics* **91**, 461-466 (1989). DOI: 10.1063/1.457480.

[4] J. Racine, "Consistent Cross-Validatory Model-Selection for Dependent Data: hv-Block Cross-Validation," *Journal of Econometrics* **99**, 39-61 (2000). DOI: 10.1016/S0304-4076(00)00030-0.

[5] D. R. Roberts, V. Bahn, S. Ciuti, et al., "Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure," *Ecography* **40**, 913-929 (2017). DOI: 10.1111/ecog.02881.

[6] J. D. Morrow, J. L. A. Gardner, and V. L. Deringer, "How to Validate Machine-Learned Interatomic Potentials," *Journal of Chemical Physics* **158**, 121501 (2023). DOI: 10.1063/5.0139611.

[7] VASP Software GmbH, "Smearing technique," VASP Wiki. Available at: https://vasp.at/wiki/Smearing_technique (accessed 2026-07-27).

[8] ACEsuit, "Training," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/training.html (accessed 2026-07-27).

[9] ACEsuit, `mace-torch` 0.3.16, Python Package Index, released 2026-05-10. Available at: https://pypi.org/project/mace-torch/0.3.16/ (accessed 2026-07-27).

[10] ACEsuit, `estimate_e0s_from_foundation`, MACE reference implementation, version-locked by the adapter at implementation time. Current source available at: https://github.com/ACEsuit/mace/blob/main/mace/data/utils.py (accessed 2026-07-27).

[11] ACEsuit, "Multihead Replay Finetuning," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/multihead_finetuning.html (accessed 2026-07-27).

[12] ACEsuit, "Multihead Training for MACE," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/multihead_training.html (accessed 2026-07-27).

[13] C. Schran, K. Brezina, and O. Marsalek, "Committee Neural Network Potentials Control Generalization Errors and Enable Active Learning," *Journal of Chemical Physics* **153**, 104105 (2020). DOI: 10.1063/5.0016004.

[14] A. R. Tan, S. Urata, S. Goldman, J. C. B. Dietschreit, and R. Gomez-Bombarelli, "Single-Model Uncertainty Quantification in Neural Network Potentials Does Not Consistently Outperform Model Ensembles," *npj Computational Materials* **9**, 225 (2023). DOI: 10.1038/s41524-023-01180-8.

[15] I. Batatia, P. Benner, Y. Chiang, et al., "A Foundation Model for Atomistic Materials Chemistry," *Journal of Chemical Physics* **163**, 184110 (2025). DOI: 10.1063/5.0297006.

[16] M. Kulichenko, B. Nebgen, N. Lubbers, J. S. Smith, et al., "Data Generation for Machine Learning Interatomic Potentials and Beyond," *Chemical Reviews* **124**, 13681-13714 (2024). DOI: 10.1021/acs.chemrev.4c00572.

[17] ACEsuit, `mace.tools.train`, MACE version 0.3.16 source, especially the validation-head iteration and last-head checkpoint rule. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/mace/tools/train.py (accessed 2026-07-27).

[18] ACEsuit, `mace.cli.run_train`, MACE version 0.3.16 source, especially multi-head assembly, replay-ratio duplication, head ordering, and loader construction. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/mace/cli/run_train.py (accessed 2026-07-27).

[19] ACEsuit, `mace-torch` version 0.3.16 `setup.cfg`, complete runtime `install_requires` contract. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/setup.cfg (accessed 2026-07-28).

[20] e3nn developers, `e3nn` 0.4.4 package metadata and dependency contract, Python Package Index. Available at: https://pypi.org/project/e3nn/0.4.4/ (accessed 2026-07-28).

[21] R. Kern, "A Simple File Format for NumPy Arrays," NumPy Enhancement Proposal 1, 2007. Available at: https://numpy.org/doc/1.13/neps/npy-format.html (accessed 2026-08-15).

[22] NumPy developers, "numpy.load," NumPy reference documentation. Available at: https://numpy.org/doc/stable/reference/generated/numpy.load.html (accessed 2026-08-15).

[23] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.

[24] R. J. Hyndman and Y. Fan, “Sample Quantiles in Statistical Packages,” *The American Statistician* **50**(4), 361–365 (1996). DOI: 10.1080/00031305.1996.10473566.

[25] J. L. Bentley, “Multidimensional Binary Search Trees Used for Associative Searching,” *Communications of the ACM* **18**(9), 509–517 (1975). DOI: 10.1145/361002.361007.

[26] SciPy developers, “scipy.spatial.cKDTree,” SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html (accessed 2026-08-15).

[27] T. F. Gonzalez, “Clustering to Minimize the Maximum Intercluster Distance,” *Theoretical Computer Science* **38**, 293–306 (1985). DOI: 10.1016/0304-3975(85)90224-5.

[28] SciPy developers, “scipy.spatial.cKDTree.query,” SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query.html (accessed 2026-08-15).

[29] SciPy developers, “scipy.stats.wasserstein_distance,” SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wasserstein_distance.html (accessed 2026-08-15).

[30] K. Jamieson and A. Talwalkar, “Non-stochastic Best Arm Identification and Hyperparameter Optimization,” *Proceedings of AISTATS*, PMLR 51:240–248, 2016. Available at: https://proceedings.mlr.press/v51/jamieson16.html.

[31] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization,” *Journal of Machine Learning Research* **18**(185), 1–52 (2018). Available at: https://www.jmlr.org/papers/v18/16-558.html.

[32] R. D. Blumofe and C. E. Leiserson, “Scheduling Multithreaded Computations by Work Stealing,” *Journal of the ACM* **46**(5), 720-748 (1999). DOI: 10.1145/324133.324234.

[33] SciPy developers, `scipy.spatial.cKDTree.query_ball_point`, SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query_ball_point.html (accessed 2026-08-17).

[34] SciPy developers, compressed sparse row/column matrix documentation, including `scipy.sparse.csr_matrix` and `scipy.sparse.csc_matrix`. Available at: https://docs.scipy.org/doc/scipy/reference/sparse.html (accessed 2026-08-17).

[35] `threadpoolctl` developers, “Python helpers to limit native thread pools,” project documentation and source. Available at: https://github.com/joblib/threadpoolctl (accessed 2026-08-17).

[36] J. Deters, J. Wu, Y. Xu, and I.-T. A. Lee, “A NUMA-Aware Provably-Efficient Task-Parallel Platform Based on the Work-First Principle,” arXiv:1806.11128 (2018). Available at: https://arxiv.org/abs/1806.11128.

[37] NumPy developers, `numpy.bincount`, NumPy reference documentation. Available at: https://numpy.org/doc/stable/reference/generated/numpy.bincount.html (accessed 2026-08-17).